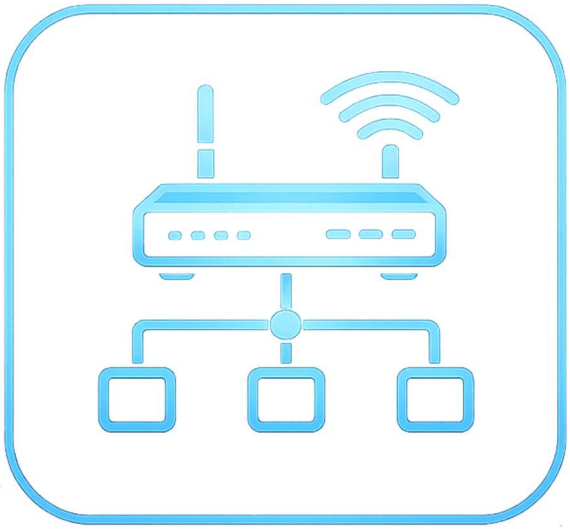


Software Routeur

Rapport de projet

Lucas Lordon
MID4 – École technique, École des métiers de Lausanne (ETML)
5 semaines – 88 heures
Du 04.02.2026 au 06.03.2026

Chef de projet : Cédric Schaffter
Expert N°1 : -
Expert N°2 : -

Table des matières

1 Analyse préliminaire 3
1.1 Introduction 3
1.2 Périmètre fonctionnel 5
1.3 Objectifs 5
1.4 Méthodologie de planification 7
1.5 Planification 7

2 Analyse / Conception 12
2.1 Architecture réseau : 12
2.2 Scripts Python : 15
2.3 Base de données : 19
2.4 Maquettes du Site 22
2.5 Stratégie de test 34
2.6 Risques techniques 36
2.7 Dossier de conception 36

3 Réalisation 37
3.1 Dossier de réalisation 37
3.2 Description des tests effectués 38
3.3 Erreurs restantes 45
3.4 Liste des documents fournis 45

4 Conclusions 45

5 Annexes 45
5.1 Résumé du rapport du TPI version succincte de la documentation 45
5.2 Glossaire 45
5.3 Webographie 46
5.4 Journal de travail 46
5.5 Manuel d'Installation 47
5.6 Manuel d'Utilisation 47
5.7 Archives du projet 47

Historique des versions

Date	Version	Nom du fichier	Description
06.02.2026	1.0	TPI_luclordon_RapportDeProjet_ETML_V1.pdf	Version initiale
13.02.2026	2.0	TPI_luclordon_RapportDeProjet_ETML_V2.pdf	Section Base de données
27.02.2026	3.0	TPI_luclordon_RapportDeProjet_ETML_V3.pdf	Section Maquettes

Tableau 1 : Historique des versions

1 Analyse préliminaire

1.1 Introduction

Ce projet a été réalisé au début de l'année 2026 durant ma 4e année de formation à l'ETML dans le cadre du CFC d'informaticien. Il correspond au Travail Pratique Individuel (TPI) obligatoire permettant de valider le CFC.

Le projet consiste en la mise en place d'un routeur logiciel destiné à un environnement domestique, ayant pour objectif de fournir un accès Internet sécurisé. Il s'inscrit dans une problématique actuelle rencontrée par de nombreux parents, à savoir la difficulté de contrôler et restreindre l'accès à certains contenus Internet pour leurs enfants, tout en conservant une utilisation simple et centralisée du réseau.

La solution proposée repose sur l'utilisation de 2 machines virtuelles sous OS Ubuntu 22.04 LTS permettant de gérer les services réseau essentiels tels que le DHCP, le DNS, le routage et le pare-feu, ainsi qu'une interface web d'administration. Cette interface permet à un utilisateur non technique (un parent) de gérer facilement des règles de filtrage Internet, en fonction des appareils du réseau domestique et des destinations à bloquer.

Ce projet permet de mettre en pratique des compétences acquises durant la formation, notamment en informatique d'entreprise, réseaux, sécurité, développement applicatif, ainsi qu'en documentation et gestion de projet.

Le temps de travail accordé pour ce projet est de 88 heures réparti sur 5 semaines, du 04.02.2026 au 06.03.2026. L'ensemble de ces heures sont réalisées dans l'enceinte de l'ETML, sur le site de Vennes. Les jours travaillés sont le lundi, mercredi et vendredi, ainsi que le jeudi après-midi.

À ce stade du projet, aucun travail préalable n’avait été réalisé. L’ensemble de la conception, de l’implémentation, des tests et de la documentation a été effectué dans le cadre du TPI, sur la base du cahier des charges fourni en début de projet.

L'évaluation de ce travail est basée sur l'ordonnance de formation entrée en vigueur en 2021. La grille d'évaluation peut être retrouvée sous : TODO. Les 3 parties qui seront évaluées ont la pondération suivante :

1) Exécution et résultat du travail 50% (23 critères)
2) Documentation : 20% (9 critères)
3) Présentation et entretien professionnel : 30% (10 critères)

Les parties 1 et 2 seront évaluées par le chef de projet Cédric Schaffter. La partie 3 sera évaluée par Expert n°1 et Expert n°2.

1.2 Périmètre fonctionnel

Le projet consiste à concevoir et mettre en place un routeur logiciel permettant de fournir un accès Internet sécurisé à un réseau domestique. Il inclut la gestion dynamique des adresses IP via un serveur DHCP, la résolution DNS avec enregistrement automatique des clients, ainsi que le filtrage des accès Internet par adresse IP ou nom de domaine.

Une interface web d'administration, protégée par authentification, permet la gestion des différentes machines du réseau domestique et des règles de filtrage associées à chacune d'elles. Les règles de sécurité sont enregistrées de manière persistante dans une base de données MySQL, modélisée selon les bonnes pratiques.

Le projet n'inclut pas la mise en place de fonctionnalités avancées telles que l'inspection du contenu chiffré HTTPS, la gestion du temps d'utilisation, la haute disponibilité, la prise en charge d'IPv6 ou la gestion multi-utilisateur.

1.3 Objectifs

L'objectif principal du projet est de concevoir, implémenter et documenter un routeur logiciel fonctionnel permettant de sécuriser l'accès Internet d'un réseau domestique.

À la fin du projet, les objectifs suivants devront être atteints et vérifiables :

Objectif général

- Mettre en place un routeur logiciel opérationnel assurant un contrôle d'accès Internet simple et sécurisé pour un réseau domestique.

Objectifs spécifiques

1. Installer et configurer deux machines virtuelles sous Ubuntu 22.04 LTS afin de fournir l'infrastructure réseau nécessaire au projet.

2. Mettre en place un serveur DHCP permettant l'attribution dynamique des adresses IP aux clients du réseau domestique avec un bail de 6 heures.

3. Mettre en œuvre un serveur DNS assurant :

- la résolution des noms de domaine vers des adresses IP,

- la résolution inverse (IP vers nom),

- l'enregistrement automatique des clients ayant obtenu une adresse IP via le DHCP.

4. Développer un mécanisme de filtrage du trafic Internet permettant de bloquer l'accès à des adresses IP, des plages d'adresses IP et des noms de domaine spécifiques pour une machine donnée du réseau local.

5. Concevoir et implémenter une interface web d'administration permettant :

- l'affichage des clients du réseau local,

- l'ajout, la modification et la suppression de règles de filtrage,

- l'association de règles à des machines spécifiques.

6. Mettre en place une authentification pour l'accès à l'interface d'administration avec une gestion de session d'une durée de 10 minutes.

7. Concevoir une base de données MySQL respectant les bonnes pratiques de modélisation afin de stocker les clients du réseau et leurs règles de filtrage, avec suppression en cascade des règles associées à un client supprimé.

8. Assurer la persistance et l'application automatique des règles de filtrage lors du fonctionnement du routeur logiciel.

9. Mettre en place un système de versioning à l'aide de GitHub, démontrant des commits réguliers, cohérents et correctement documentés.

10. Rédiger l'ensemble des livrables requis, incluant le rapport de projet, le journal de travail et le manuel utilisateur.

Évolution des objectifs

Les objectifs définis dans ce chapitre pourront être ajustés ou précisés à l'issue de la phase d'analyse, dans la mesure où ces modifications restent conformes au cahier des charges initial et sont validées par les experts.

1.4 Méthodologie de planification

La planification du projet a été réalisée selon la méthode des 6 étapes (IPDRCE), qui permet d'organiser les tâches de manière structurée et cohérente. Chaque tâche du projet suit les étapes suivantes :

1. S'informer

Collecter toutes les informations nécessaires sur la tâche : contexte, compétences requises, matériel et temps nécessaire. Si besoin, la tâche peut être décomposée en sous-tâches plus simples.

2. Planifier

Définir les priorités et les échéances, organiser le travail dans le temps, répartir les ressources et anticiper les risques.

3. Décider

Choisir les méthodes, outils et technologies adaptés à la réalisation de la tâche, et valider les solutions envisagées.

4. Réaliser

Mettre en œuvre la tâche selon les méthodes et outils définis, en suivant l'avancement et en ajustant si nécessaire.

5. Contrôler

Vérifier que la tâche répond aux objectifs et aux critères de qualité, identifier les écarts par rapport au plan et corriger si besoin.

6. Évaluer

Analyser les résultats, recueillir les retours et les leçons apprises, et confirmer la fin de la tâche pour améliorer les prochaines étapes du projet.

1.5 Planification

La planifiaction a été réaliser sur le logiciel Jira d'athlasian elle est disponible sous ce lien : https://p-appro-1.atlassian.net/, il faudra attendre que votre demande soit accepter.
Pour ce projet de TPI j'ai pris la décision de le séparer en 5 miles stones importante :

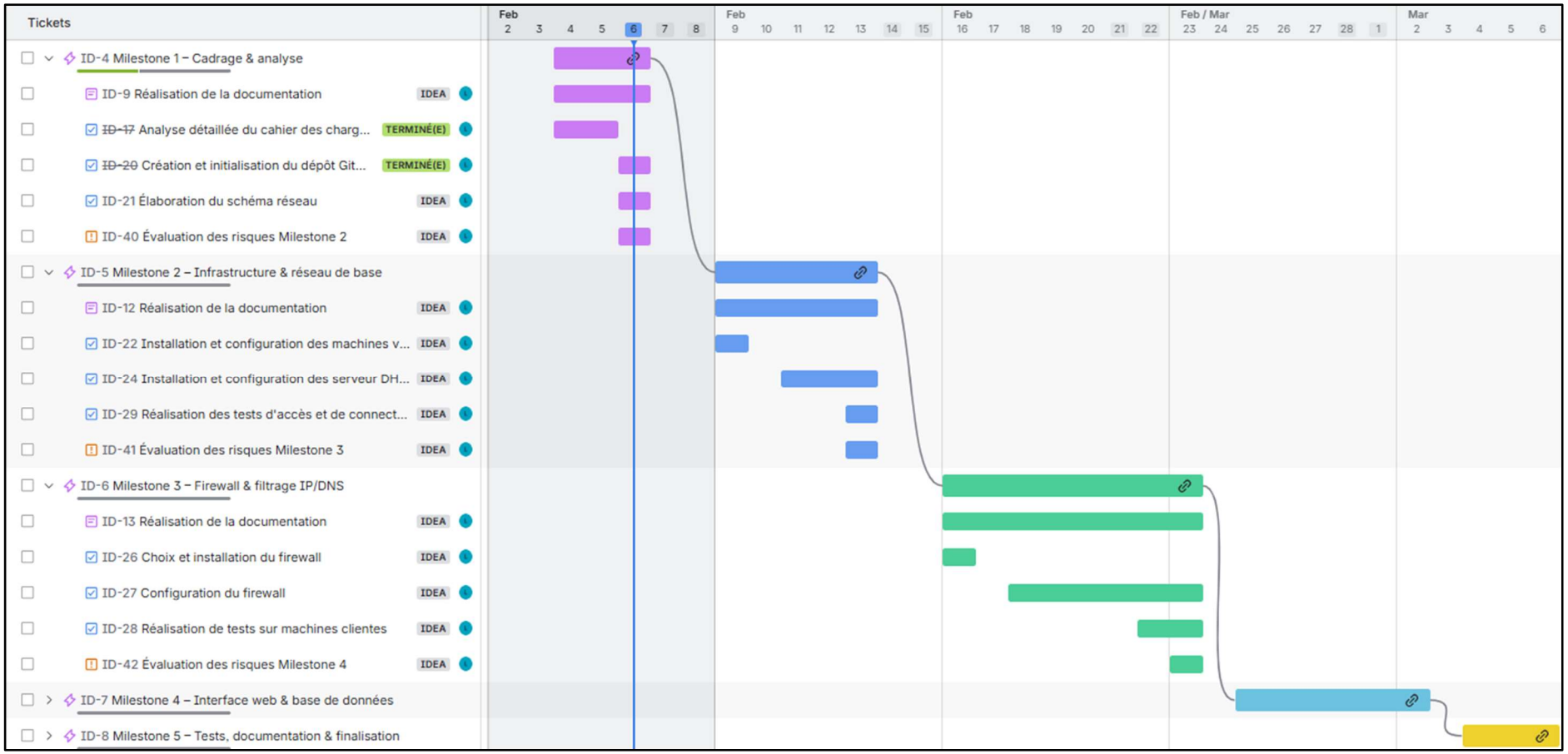


Figure 1, Kanban Milestone n°1-3

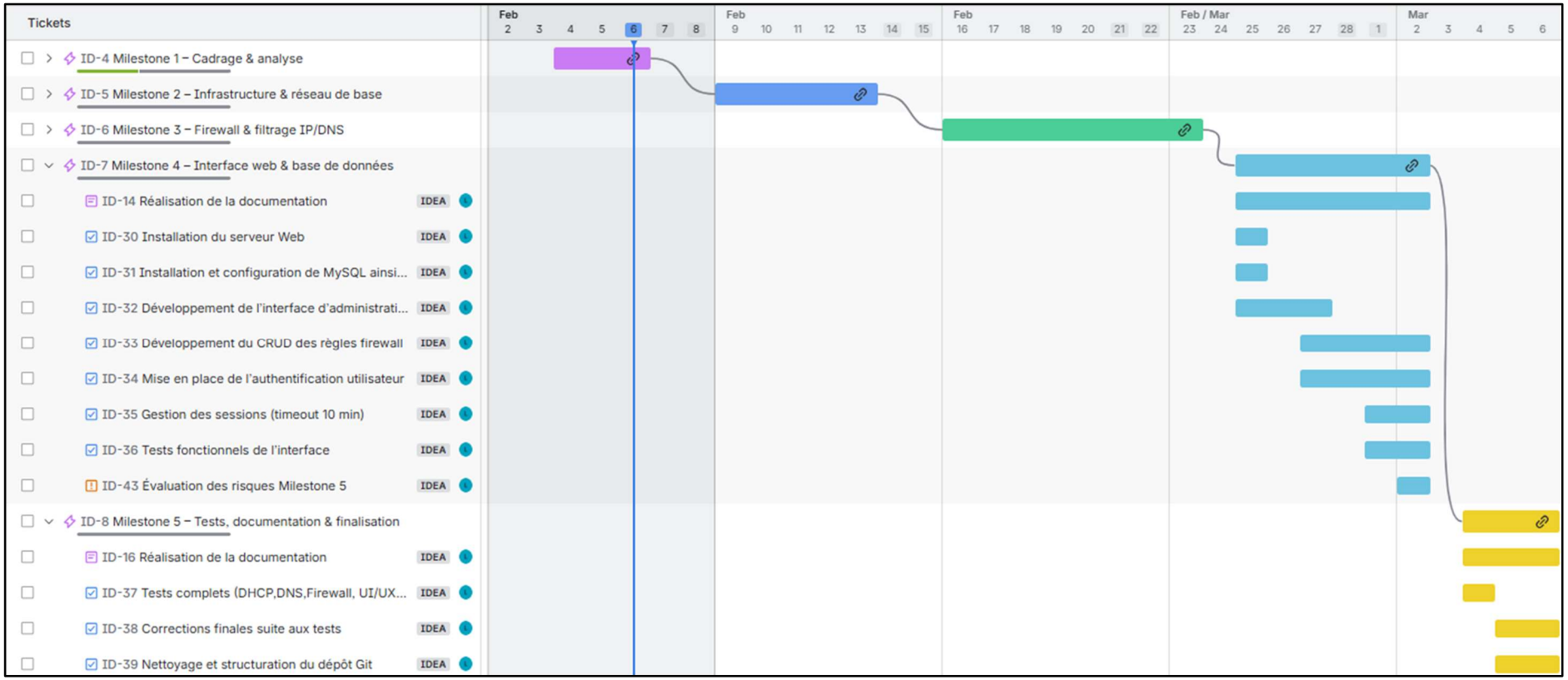


Figure 2, Kanban Milestone n°4-5

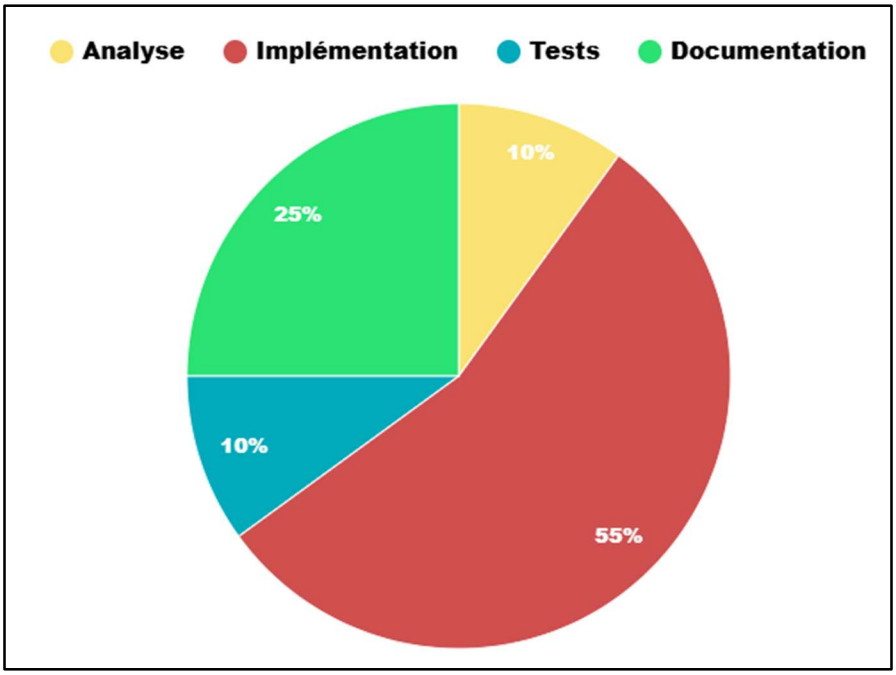


Figure 3, Répartition du travail selon planification initiale

Tickets	Start date ↑	Date d'échéance
Milestone 1 – Cadrage & analyse	04 févr. 2026	06 févr. 2026
Analyse détaillée du cahier des charges et de la grille TPI	04 févr. 2026	05 févr. 2026
Réalisation de la documentation	04 févr. 2026	06 févr. 2026
Élaboration du schéma réseau	06 févr. 2026	06 févr. 2026
Évaluation des risques Milestone 2	06 févr. 2026	06 févr. 2026
Création et initialisation du dépôt GitHub	06 févr. 2026	06 févr. 2026

Figure 4, Tableau Milestone n°1

Tickets	Start date ↑	Date d'échéance
Milestone 2 – Infrastructure & réseau de base	09 févr. 2026	13 févr. 2026
Réalisation de la documentation	09 févr. 2026	13 févr. 2026
Installation et configuration des machines virtuelles Ubu...	09 févr. 2026	09 févr. 2026
Installation et configuration des serveur DHCP et DNS	11 févr. 2026	13 févr. 2026
Réalisation des tests d'accès et de connectivité	13 févr. 2026	13 févr. 2026
Évaluation des risques Milestone 3	13 févr. 2026	13 févr. 2026

Figure 5, Tableau Milestone n°2

Tickets	Start date	Date d'échéance
Milestone 3 – Firewall & filtrage IP/DNS	16 févr. 2026	23 févr. 2026
Réalisation de la documentation	16 févr. 2026	23 févr. 2026
Choix et installation du firewall	16 févr. 2026	16 févr. 2026
Configuration du firewall	18 févr. 2026	23 févr. 2026
Réalisation de tests sur machines clientes	22 févr. 2026	23 févr. 2026
Évaluation des risques Milestone 4	23 févr. 2026	23 févr. 2026

Figure 6, Tableau Milestone n°3

Tickets	Start date	Date d'échéance
Milestone 4 – Interface web & base de données	25 févr. 2026	02 mars 2026
Réalisation de la documentation	25 févr. 2026	02 mars 2026
Installation du serveur Web	25 févr. 2026	25 févr. 2026
Installation et configuration de MySQL ainsi que la DB (d...	25 févr. 2026	25 févr. 2026
Développement de l'interface d'administration	25 févr. 2026	27 févr. 2026
Développement du CRUD des règles firewall	27 févr. 2026	02 mars 2026
Mise en place de l'authentification utilisateur	27 févr. 2026	02 mars 2026
Gestion des sessions (timeout 10 min)	01 mars 2026	02 mars 2026
Tests fonctionnels de l'interface	01 mars 2026	02 mars 2026
Évaluation des risques Milestone 5	02 mars 2026	02 mars 2026

Figure 7, Tableau Milestone n°4

Tickets	Start date	Date d'échéance
Milestone 5 – Tests, documentation & finalisation	04 mars 2026	06 mars 2026
Tests complets (DHCP,DNS,Firewall, UI/UX, auth web)	04 mars 2026	04 mars 2026
Réalisation de la documentation	04 mars 2026	06 mars 2026
Corrections finales suite aux tests	05 mars 2026	06 mars 2026
Nettoyage et structuration du dépôt Git	05 mars 2026	06 mars 2026

Figure 8, Tableau Milestone n°5

2 Analyse / Conception

2.1 Architecture réseau :

Contraintes du cahier des charges :

Le cahier des charges impose la mise en place d'une infrastructure réseau virtualisée composée de deux machines virtuelles Ubuntu. La première machine (VM1) doit héberger les services DHCP et DNS pour le réseau interne. La seconde (VM2) doit assurer le rôle de routeur logiciel avec filtrage par pare-feu, héberger la base de données MySQL ainsi que l'application web d'administration. Cette application doit être accessible non seulement depuis le réseau interne, mais également depuis un appareil extérieur à VirtualBox connecté au même réseau physique (par exemple un téléphone ou le PC hôte).

Le CDC spécifie également que les clients du réseau interne doivent obtenir automatiquement leur configuration réseau (adresse IP, passerelle, serveur DNS) via DHCP, avec un bail de 6 heures. Le domaine interne est défini comme « safehome.local ». Enfin, le filtrage doit pouvoir bloquer ou autoriser l'accès à des destinations spécifiques (domaines, adresses IP ou plages CIDR) par client, via des règles du firewall gérées depuis l'interface web.

Décisions d'architecture :

Séparation des rôles sur deux VM :

La répartition des services sur deux machines virtuelles distinctes reflète un principe de séparation des responsabilités. VM1 se concentre exclusivement sur l'approvisionnement réseau (DHCP et DNS), tandis que VM2 centralise le contrôle du trafic et la gestion applicative. Cette séparation présente plusieurs avantages : elle isole les services critiques d'attribution d'adresses du reste de l'infrastructure, elle simplifie le dépannage en cas de problème, et elle reproduit une architecture que l'on retrouve dans des environnements réels où les serveurs DHCP/DNS sont souvent dédiés.

Choix du sous-réseau 192.168.15.0/24 :

Le sous-réseau 192.168.15.0/24 a été choisi pour le réseau interne afin d'éviter tout conflit avec le réseau physique du FAI (généralement en 192.168.1.0/24 ou 192.168.0.0/24). La plage DHCP a été définie de 192.168.15.100 à 192.168.15.200, laissant les adresses basses disponibles pour les équipements à IP fixe. VM2 occupe l'adresse 192.168.15.1 (passerelle par défaut du réseau) et VM1 l'adresse 192.168.15.254 (serveur DNS et DHCP).

Mode réseau VirtualBox : Internal Network + Bridged Adapter :

Le réseau interne VirtualBox (« intnet-lan ») connecte les trois machines (VM1, VM2 et la VM Client de test) dans un réseau isolé, simulant un LAN physique. VirtualBox

agit comme un switch Ethernet virtuel entre les interfaces enp0s3 de chaque VM connectée à ce réseau.

VM2 dispose d'une seconde interface réseau (enp0s8) configurée en mode « Bridged Adapter ». Ce mode permet à VM2 d'obtenir une adresse IP directement sur le réseau physique du FAI, comme si elle était une machine supplémentaire connectée au routeur domestique. C'est cette interface qui assure deux fonctions essentielles : la sortie vers Internet (via NAT/MASQUERADE pour les clients internes) et l'accessibilité de l'application web depuis le PC hôte ou tout autre appareil du réseau local physique.

NAT par MASQUERADE sur VM2 :

VM2 assure le routage entre le réseau interne (enp0s3) et Internet (enp0s8) grâce à l'activation de l'IP forwarding dans le noyau Linux et à une règle iptables MASQUERADE dans la chaîne POSTROUTING de la table NAT. Le choix de MASQUERADE plutôt que SNAT s'explique par le fait que l'adresse IP de l'interface bridge (enp0s8) est attribuée dynamiquement par le DHCP du FAI et peut donc changer. MASQUERADE adapte automatiquement la traduction d'adresse source à l'IP courante de l'interface sortante, ce qui évite de devoir modifier la règle à chaque changement d'adresse.

DNS dynamique (DDNS) entre DHCP et BIND9 :

Le serveur DHCP (isc-dhcp-server) et le serveur DNS (BIND9), tous deux sur VM1, sont configurés pour fonctionner ensemble via le protocole DDNS (Dynamic DNS). Lorsqu'un client obtient un bail DHCP, le serveur DHCP notifie automatiquement BIND9 pour créer un enregistrement DNS correspondant dans la zone « safehome.local ». Cela permet de résoudre les noms d'hôtes des clients sans configuration manuelle, ce qui est particulièrement utile pour identifier les appareils par leur nom plutôt que par leur adresse IP.

Déploiement de l'application web avec Kestrel:

L'application web est basé sur ASP.NET Core et est déployée directement sur VM2 en utilisant le, sans conteneurisation Docker. Ce choix a été fait pour simplifier le déploiement dans le cadre du TPI et éviter une couche de complexité supplémentaire. L'application écoute sur toutes les interfaces (0.0.0.0:5000), ce qui la rend accessible à la fois depuis le réseau interne (http://192.168.15.1:5000) et depuis le réseau physique via le bridge (http://<IP-bridge-VM2>:5000). Un service

systemd assure le démarrage automatique de l'application et son exécution en tant que root, nécessaire pour l'exécution des commandes iptables.

Description du schéma réseau :

Le schéma réseau présente l'architecture complète de l'infrastructure SafeHome. L'ensemble est contenu dans un rectangle représentant le PC hôte Windows 10 exécutant VirtualBox :

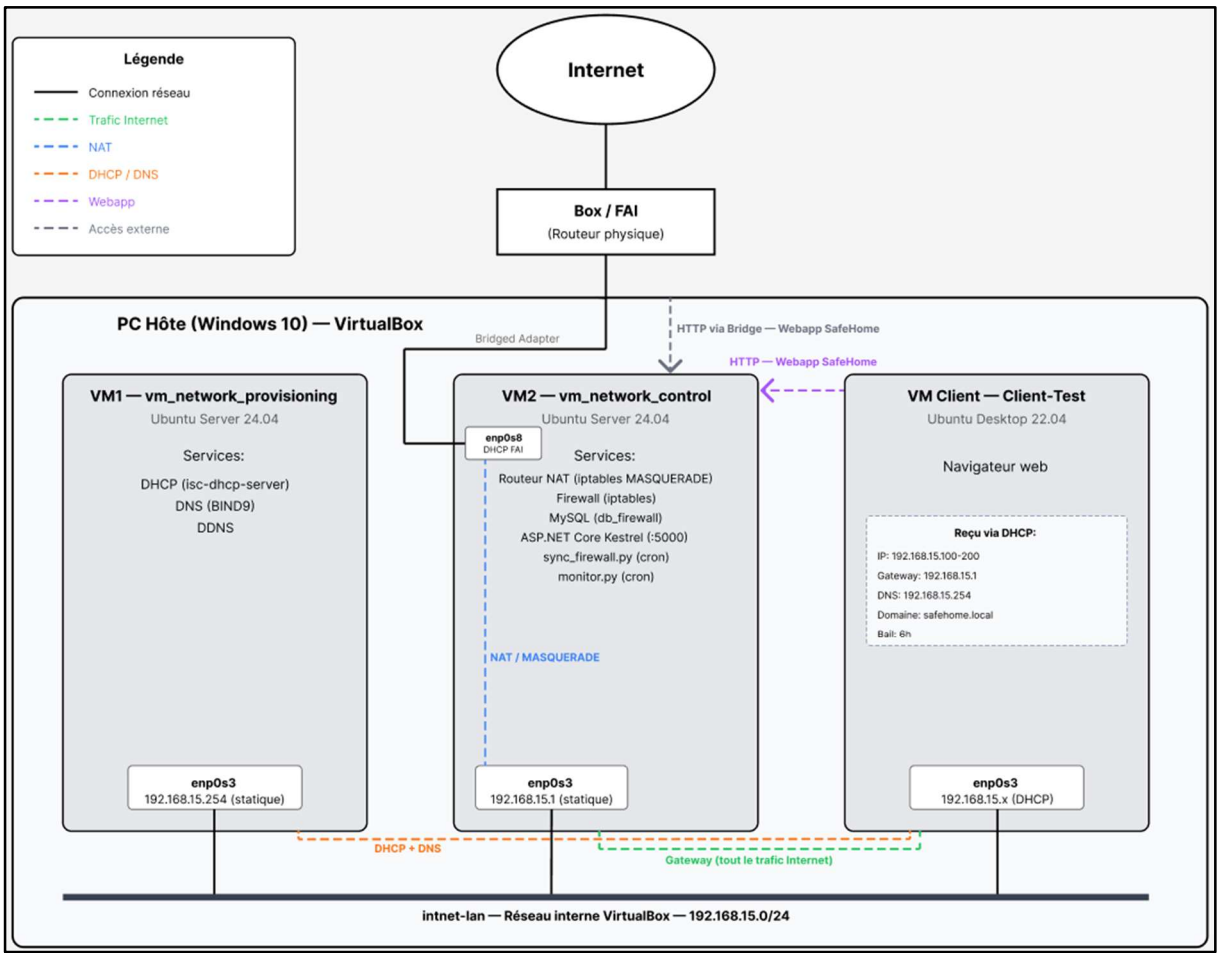


Schéma 1 : Architecture du réseau

Au centre du schéma, les trois machines virtuelles sont disposées côte à côte. VM1 (à gauche) héberge les services DHCP, DNS et DDNS avec une seule interface réseau sur le réseau interne (192.168.15.254). VM2 (au centre) concentre les services de routage NAT, de filtrage iptables, la base de données MySQL, le serveur web ASP.NET Core Kestrel, ainsi que les scripts Python de synchronisation des règles (sync_firewall.py) et de monitoring (monitor.py). VM2 possède deux interfaces : enp0s3 sur le réseau interne (192.168.15.1) et enp0s8 en bridge vers le réseau physique. La VM Client (à droite) représente un poste utilisateur avec un navigateur web, configuré automatiquement via DHCP.

En bas du schéma, une barre horizontale représente le réseau interne VirtualBox « intnet-lan » (192.168.15.0/24), auquel les trois VM sont connectées via leur interface enp0s3. Toutes les machines partagent le même segment de couche 2 (Ethernet) ; c'est la configuration IP (passerelle, DNS) qui détermine les flux de couche 3.

En haut du schéma, la connexion vers Internet passe par l'interface bridge de VM2 (enp0s8), qui rejoint le routeur du FAI (Box) puis Internet. C'est également par cette interface bridge que le PC hôte peut accéder à l'application web.

Les flux logiques sont représentés par des flèches pointillées de couleurs différentes : vert pour le trafic Internet transitant par la gateway (VM2), orange pour les requêtes DHCP et DNS vers VM1, violet pour l'accès à la webapp SafeHome, bleu pour la traduction NAT/MASQUERADE à l'intérieur de VM2, et gris pour l'accès depuis le PC hôte via le bridge.

2.2 Scripts Python :

Pourquoi des scripts Python séparés de l'application web :

L'application web ASP.NET Core gère l'interface d'administration : l'utilisateur y crée, modifie et supprime des règles de filtrage et des clients via le navigateur. Ces opérations écrivent dans la base de données MySQL. Cependant, deux besoins fondamentaux ne peuvent pas être couverts par l'application web seule.

Le premier besoin concerne l'application des règles sur le système. Lorsqu'un administrateur ajoute une règle de filtrage via l'interface, cette règle est enregistrée en base de données mais n'a aucun effet réel sur le trafic réseau tant qu'elle n'a pas été traduite en commande iptables sur la VM2. Il faut donc un mécanisme qui lit les règles en base et les applique dans la chaîne iptables correspondante. Ce mécanisme doit également fonctionner dans l'autre sens : si une règle est supprimée en base, la règle iptables associée doit disparaître.

Le second besoin concerne la collecte de données en temps réel. Le dashboard de l'application affiche des indicateurs dynamiques : quels clients sont actuellement connectés au réseau (online/offline), combien de blocages ont eu lieu aujourd'hui, quels domaines ont été bloqués pour quel client. Ces informations n'existent pas dans la base de données par défaut — elles doivent être collectées activement depuis le système (ping des clients, lecture des logs).

La solution retenue est de déléguer ces deux responsabilités à des scripts Python indépendants, exécutés à intervalles réguliers par le planificateur de tâches cron de Linux. Ce choix présente plusieurs avantages. Python offre une interaction directe et simple avec les commandes système (subprocess), la base de données MySQL (pymysql) et le parsing de fichiers texte (regex). Les scripts fonctionnent indépendamment de l'application web : même si celle-ci est redémarrée ou temporairement indisponible, la synchronisation des règles et le monitoring continuent. Enfin, la séparation des responsabilités rend chaque composant plus simple à comprendre, tester et maintenir.

Sync-firewall.py - Synchronisation des règles :

Objectif et logique :

Le script sync_firewall.py assure la correspondance entre l'état souhaité (les règles stockées en base de données) et l'état réel (les règles iptables actives sur le système). Il est exécuté par cron toutes les 5 minutes.

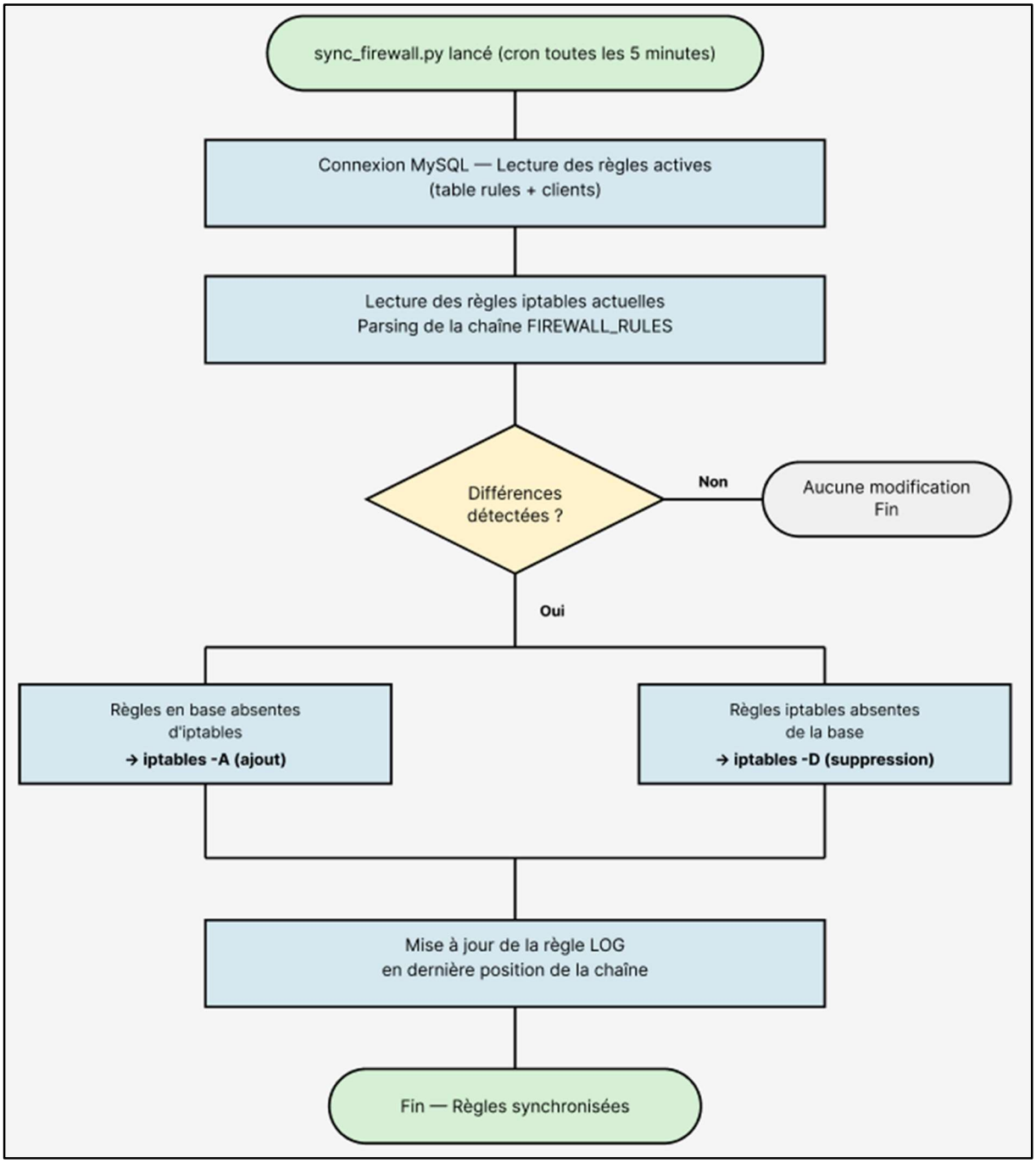


Schéma 2 : Script Sync_firewall.py

À chaque exécution, le script commence par lire l'ensemble des règles actives depuis MySQL en joignant les tables « firewall_rules » et « clients » pour obtenir à la fois la destination à bloquer (domaine, IP ou CIDR) et l'adresse IP du client concerné. En parallèle, il interroge iptables pour obtenir la liste des règles actuellement présentes dans la chaîne dédiée « FIREWALL_RULES ».

Le script compare ensuite ces deux ensembles. Si des règles présentes en base n'existent pas encore dans iptables, elles sont ajoutées (iptables -A). Si des règles présentes dans iptables n'ont plus de correspondance en base (parce que l'administrateur les a supprimées via l'interface), elles sont retirées (iptables -D). Cette approche évite de vider et recréer l'intégralité des règles à chaque exécution, ce qui pourrait provoquer une brève interruption du filtrage.

Pour les règles de type « domaine », le script effectue une résolution DNS afin d'obtenir les adresses IP correspondantes, car iptables ne travaille qu'avec des adresses IP. Les IPs résolues sont stockées dans la table « firewall_logs » pour permettre leur réutilisation ultérieure, notamment par le script de monitoring.

Avant chaque règle DROP, le script insère une règle LOG avec le préfixe « [FW_BLOCKED] ». Ce préfixe unique permet au script de monitoring d'identifier précisément les paquets bloqués par les règles utilisateur, en les distinguant du trafic loggé par d'autres chaînes iptables (NAT, FORWARD, etc.).

En fin de chaîne, la règle LOG générale est repositionnée en dernière position pour garantir que le logging des blocages intervient toujours avant le DROP et que l'ordre des règles reste cohérent.

Données produites :

Le script produit deux résultats essentiels pour le projet. D'une part, l'état iptables du système reflète fidèlement les décisions de l'administrateur prises dans l'interface web. D'autre part, la table « firewall_logs » conserve un historique des synchronisations effectuées, incluant les IPs résolues pour chaque domaine, le nombre de règles appliquées et les éventuelles erreurs. Ces logs de synchronisation sont consultables depuis le dashboard et permettent à l'administrateur de vérifier que ses règles sont bien en vigueur.

monitor.py - Monitoring et détection des blocages :

Objectif et logique :

Le script monitor.py collecte les données de supervision du réseau. Il est exécuté par cron toutes les 2 minutes et accomplit deux tâches distinctes lors de chaque

exécution.

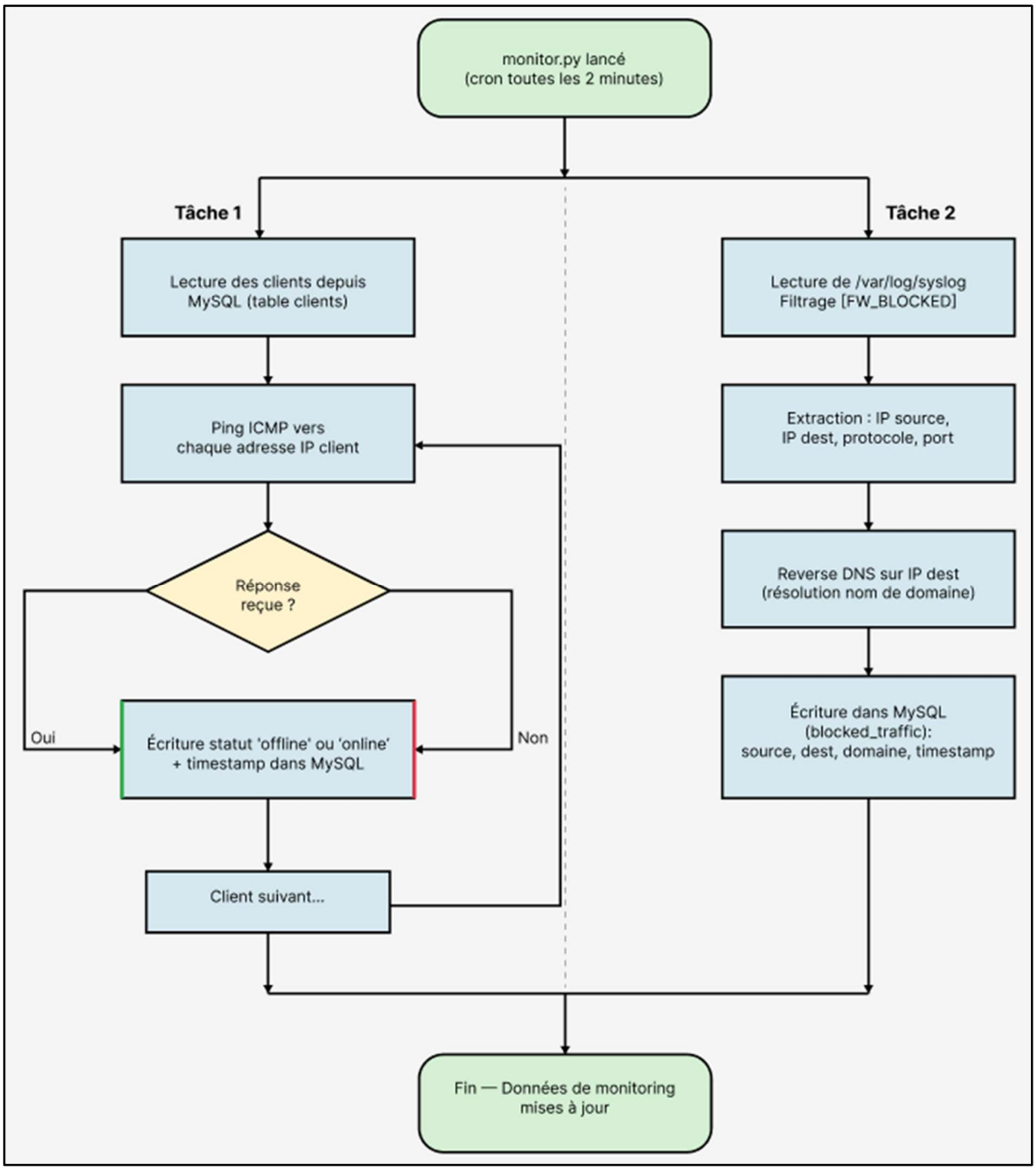


Schéma 3 : Script monitor.py

La première tâche vérifie la connectivité de chaque client enregistré dans la base de données. Le script lit la liste des clients (hostname et adresse IP) depuis la table « clients », puis envoie un ping ICMP vers chaque adresse. Selon la réponse obtenue, il enregistre le statut « online » ou « offline » dans la table « monitoring » avec un timestamp. Ce mécanisme permet au dashboard d'afficher en temps réel quels appareils du réseau sont connectés, ce qui est une information essentielle pour un outil de gestion de réseau domestique.

La seconde tâche est de détecte et enregistre le trafic bloqué par le pare-feu. Le script lit le fichier /var/log/syslog en recherchant les lignes contenant le préfixe « [FW_BLOCKED] » inséré par sync_firewall.py. Pour chaque ligne trouvée, il extrait les champs pertinents : adresse IP source (le client qui a tenté l'accès), adresse IP destination (le serveur bloqué), le protocole et le port de destination. Étant donné qu'une seule tentative d'accès à un site web génère des dizaines de paquets (DNS, TCP SYN, retransmissions), le script regroupe les entrées par combinaison « IP source + IP destination + minute » pour éviter de saturer la base avec des doublons. Un compteur (hit_count) indique le nombre réel de paquets interceptés pour chaque entrée regroupée.

Pour rendre les données exploitables dans l'interface web, le script tente de résoudre chaque IP destination en nom de domaine. Il consulte d'abord la table « firewall_logs » qui contient les associations IP-domaine issues de la synchronisation des règles, c'est la source la plus fiable. En fallback, il effectue un reverse DNS classique. Le domaine résolu est stocké dans la colonne « domain » de la table « blocked_traffic », ce qui permet au dashboard d'afficher « facebook.com » plutôt qu'une adresse IP brute.

Pour éviter de retraiter les mêmes lignes de syslog à chaque exécution, le script utilise un fichier de position qui mémorise l'emplacement exact dans le fichier syslog où la dernière lecture s'est arrêtée. À chaque exécution, il reprend la lecture à partir de cette position. Le script gère également la rotation de syslog (logrotate) en vérifiant l'inode du fichier : si celui-ci a changé, la lecture reprend depuis le début du nouveau fichier.

Données produites :

Les données générées par monitor.py alimentent directement les indicateurs du dashboard de l'application web. La table « monitoring » fournit le nombre de clients en ligne, le statut de chaque appareil et le nombre de règles actives à un instant donné. La table « blocked_traffic » fournit le nombre de blocages du jour (affiché dans la carte de statistiques du dashboard), le détail des tentatives d'accès bloquées par client (consultable pour comprendre quels sites ont été filtrés), et les domaines associés aux IPs bloquées (pour une lecture humaine des données). Sans ces données, le dashboard ne pourrait afficher que des informations statiques (liste des clients, liste des règles) et perdrait sa valeur principale : offrir une vision en temps réel de l'activité du réseau et de l'efficacité du filtrage.

2.3 Base de données :

Une base de données MySQL doit être créée et hébergée sur la VM2. Les tables liées aux scripts Python (monitoring, blocked_traffic et firewall_logs), qui n'avaient pas été anticipées lors de l'analyse initiale de la base de données, seront détaillées dans la partie réalisation.

Les contraintes imposées pour la conception initiale de la base de données sont les suivantes :

1) La base de données doit être une base MySQL nommée db_firewall.
2) Les règles de filtrage doivent être enregistrées de manière persistante dans la base de données.
3) La base de données doit être modélisée par l'apprenti.
4) La conception doit respecter les bonnes pratiques de modélisation (normalisation, relations claires, intégrité référentielle).
5) Un client du réseau est identifié par :
- un nom
- une adresse IP
6) Un client peut :
- avoir une ou plusieurs règles
- ne pas avoir de règle
7) Une règle de filtrage :
- concerne un seul et unique client
- ne peut pas exister sans client
8) Lorsqu'un client est supprimé, toutes les règles associées doivent être supprimées automatiquement (suppression en cascade).
9) Les règles doivent permettre le filtrage :
- par nom de domaine
- par adresse IP unique
- par plage d'adresses IP (CIDR)
10) Les données doivent pouvoir être consultées et modifiées via la page d'administration web.
11) La base de données doit être compatible avec un système d'authentification pour l'administration.

Justification du schéma :

Le schéma a été conçu à partir des contraintes métiers définis dans le cahier des charges.
La séparation entre les clients, les règles de filtrage et les utilisateurs administrateurs respecte le principe de responsabilité unique et garantit une bonne évolutivité du système.

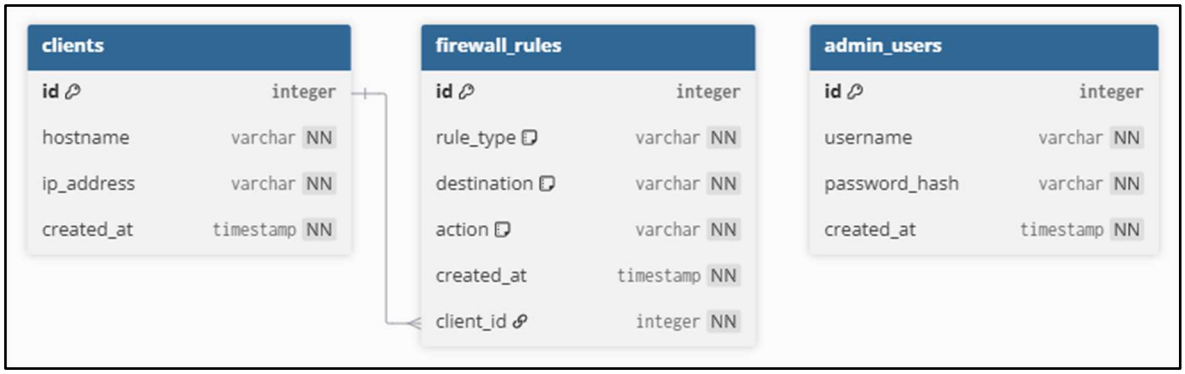


Figure 9, Schémas de la conception de base de données

Le modèle de la base de données db_firewall comporte trois tables principales :

clients :

La table clients représente l'ensemble des machines présentes sur le réseau domestique (192.168.15.0/24).
Ces machines obtiennent une adresse IP dynamiquement via le serveur DHCP configuré sur la VM1.

Chaque client peut posséder zéro, une ou plusieurs règles de filtrage.

firewall_rules :

Cette table contient l'ensemble des règles de filtrage appliquées aux différents clients.
Elle est liée à la table clients par une clé étrangère.

Les champs ont été conçus afin de faciliter le traitement côté pare-feu (iptables), notamment pour permettre le filtrage par domaine, IP unique ou plage CIDR.

Une contrainte de suppression en cascade est appliquée afin d'assurer l'intégrité des données.

admin_users :

Cette table permet la gestion des comptes administrateurs de l'interface web.
Elle est indépendante des clients et des règles afin de séparer la logique d'authentification de la logique métier du firewall.

Les trois tables sont composées de tous les champs ci-dessous :

Nom de la table	Nom du champ	Type	Contraintes	Justification / Problème résolu
clients	id	INT	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique clients.
clients	name	VARCHAR(100)	NOT NULL	Permet d'identifier facilement la machine (ex: PC-Salon).
clients	ip_address	VARCHAR(45)	NOT NULL, UNIQUE	Stocke l'adresse IPv4 ou IPv6. UNIQUE évite les doublons.
clients	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Permet de tracer l'ajout du client.
firewall_rules	id	INT	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique de la règle.

firewall_rules	client_id	INT	FOREIGN KEY REFERENCES clients(id) ON DELETE CASCADE	Assure qu'une règle appartient à un seul client. Suppression automatique si le client est supprimé.
firewall_rules	rule_type	ENUM('DOMAIN', 'IP','CIDR')	NOT NULL	Permet de différencier le type de filtrage. Simplifie le traitement iptables.
firewall_rules	value	VARCHAR(255)	NOT NULL	Contient le domaine, l'IP ou la plage CIDR à bloquer.
firewall_rules	action	ENUM('ALLOW', 'BLOCK')	NOT NULL	Permet de définir le comportement du firewall.
firewall_rules	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Traçabilité des règles ajoutées.
admin_users	id	INT	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique administrateur.
admin_users	username	VARCHAR(100)	NOT NULL, UNIQUE	Empêche les doublons de comptes.
admin_users	password_hash	VARCHAR(255)	NOT NULL	Stockage sécurisé du mot de passe (hashé).
admin_users	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Permet un suivi des comptes créés.

Tableau 2 : Justification des champs de la Base de données "db_firewall"

2.4 Maquettes du Site

Approche de conception :

Avant de commencer le développement de l'interface web, j'ai réalisé des maquettes basse fidélité pour chaque page de l'application "SafeHome". L'objectif était de définir la structure, la navigation et les interactions principales avant d'écrire la moindre ligne de code. Cette étape m'a permis de valider l'ergonomie générale et de gagner du temps lors de l'implémentation.Il faut tout de même noter que le design est minimaliste étant donné qu'il ne fait pas partie des objectifs évalués. Le mot d'ordre principal pour le design sera donc la "Simplicité ».

Les maquettes ont été conçues en gardant à l'esprit le public cible de l'application : un parent ou un administrateur réseau domestique qui souhaite gérer les règles de filtrage Internet de manière simple et intuitive. L'interface devait donc être claire, fonctionnelle et accessible sans formation technique préalable.

Il est important de préciser que l'ensemble des maquettes a été réalisé à l'aide de l'outil Figma. Cet outil permet de concevoir des interfaces variées et professionnelles, tout en facilitant la structuration des idées. Le choix de Figma

s’explique par la maîtrise approfondie que j’ai développée au cours de mes quatre années de formation. De plus, une utilisation rigoureuse de cet outil permet un gain de temps considérable pour les étapes suivantes du projet, notamment lors du développement et des éventuelles modifications.

Structure de la navigation :

Le site est organisé autour de trois pages principales, accessibles via la navbar dans l'en-tête ainsi que d’une page de login :

Dashboard : C’est la page d'accueil, elle offre une vue d'ensemble du réseau domestique. Elle affiche quatre indicateurs clés (nombre total de clients, clients en ligne, règles actives et nombre de blocages du jour) sous forme de cartes de statistiques.
En dessous de ces « scorecard », un tableau présent l'ensemble des clients du réseau avec leur statut de connectivité en temps réel (online/offline). Un lien permet d'accéder directement à la page de gestion des clients.

Edit Users : Cette page permet la gestion des clients (appareils) du réseau. Elle présente la liste des appareils enregistrés avec leur hostname et leur adresse IP dans un tableau. Des boutons d'action permettent de modifier ou supprimer chaque client, ainsi que d'ajouter un nouvel appareil. Finalement, un bouton supplémentaire permet de visualiser directement les règles associées à un client spécifique.

Edit Rules : La page principale de l'application permet de gérer les règles de filtrage du pare-feu. Elle intègre une barre de recherche, des filtres par type de règle (domaine, IP, CIDR) et par action (deny, allow), ainsi qu'un tableau triable affichant toutes les règles existantes. Un compteur de résultats qui s'adapte dynamiquement aux filtres appliqués est également présent. Les types et actions sont présentés sous forme de badges colorés pour faciliter la lecture rapide.

Login : La page de login est une page autonome avec une mise en page centrée, affichant le logo et le nom de l'application, suivis d'un formulaire de connexion simple (nom d'utilisateur et mot de passe). Elle ne partage pas le layout des autres pages puisque l'utilisateur n'est pas encore authentifié.

Principes de design :

Plusieurs principes ont guidé la conception des maquettes afin de garder une cohérence logique et visuelle sur l'ensemble de l'application :

Cohérence visuelle : Toutes les pages de l’application partagent le même gabarit (layout), comprenant un en-tête commun avec le logo SafeHome, les onglets de navigation, le nom de l’utilisateur connecté ainsi qu’un bouton de déconnexion.

Afin de faciliter la création des maquettes et de garantir une cohérence visuelle constante, j’ai défini un *Design System* ainsi qu’un *Style Guide*. Ces éléments ont également été conçus dans Figma, ce qui m’a permis d’optimiser le temps de création et d’assurer une uniformité graphique sur l’ensemble des pages.

Ces deux composantes constituent la base graphique et logique du projet. Elles sont essentielles, car elles définissent l'identité visuelle de l'application et contribuent à offrir une expérience utilisateur claire et agréable. Elles représentent les fondations nécessaires pour aboutir à un design cohérent.

Dans le cadre d'un projet, lorsque ces supports sont correctement définis, partagés avec les équipes et maintenus à jour, ils permettent un gain de temps considérable et améliorent significativement l'efficacité globale du développement.

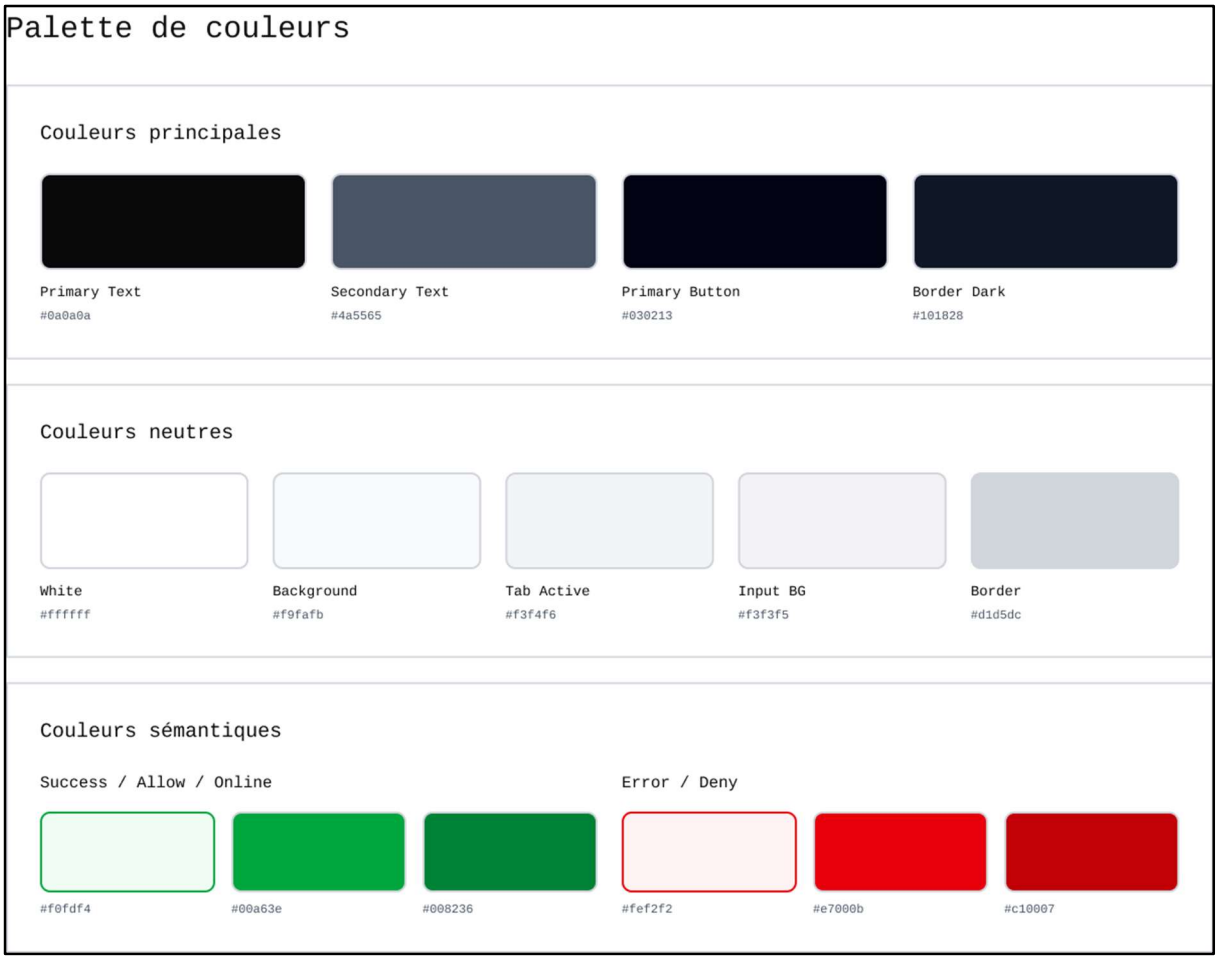
Dans la section suivante, les choix techniques relatifs à la définition du *Design System* et du *Style Guide* seront expliqués et détaillés. Pour rappel, le mot d'ordre du design de ce projet est la « Simplicité ».

Design System et Style Guide :

Palette de couleurs :

La palette s'organise en trois familles. Les couleurs principales (noirs et gris bleutés foncés) servent au texte et aux boutons d'action. Les couleurs neutres (gamme de gris clairs) définissent les arrière-plans, les bordures et les zones de saisie, créant une hiérarchie visuelle subtile sans recourir à des teintes vives.

Les couleurs sémantiques sont les plus importantes fonctionnellement : le vert est associé aux états positifs (client en ligne, règle autorisée, succès) et le rouge aux états négatifs (blocage, erreur, suppression). Ce code couleur universel permet à l'administrateur d'identifier instantanément la nature d'une information d'un simple coup d'œil sur un tableau.



Design 1 : Design System & Style Guide, Pallette de couleurs

Typographie :

Le choix repose sur deux polices complémentaires. Cousine, une police monospace, est utilisée pour les titres et les données techniques (adresses IP, noms de domaine, tableaux). Le monospace est particulièrement adapté à une application réseau car chaque caractère occupe la même largeur, ce qui aligne naturellement les données et facilite la lecture.

Arimo, une police sans-serif proportionnelle, est réservée au corps de texte, aux boutons et aux labels. Cette combinaison crée un contraste clair entre les données techniques et les éléments d'interface, tout en renforçant l'identité « administration réseau » de l'application.

Typographie

Cousine (Titres & Code)

Heading 1 - 30px / 36px
font-family: Cousine | font-size: 30px | line-height: 36px

Brand Title - 24px / 32px
font-family: Cousine | font-size: 24px | line-height: 32px

Heading 2 - 20px / 28px
font-family: Cousine | font-size: 20px | line-height: 28px

Table Data - 16px / 24px
font-family: Cousine | font-size: 16px | line-height: 24px

Table Header Bold - 14px / 20px
font-family: Cousine Bold | font-size: 14px | line-height: 20px

Badge Text - 12px / 16px
font-family: Cousine | font-size: 12px | line-height: 16px

Arimo (Corps de texte)

Paragraph - 16px / 24px
font-family: Arimo | font-size: 16px | line-height: 24px

Button & Label - 14px / 20px
font-family: Arimo | font-size: 14px | line-height: 20px

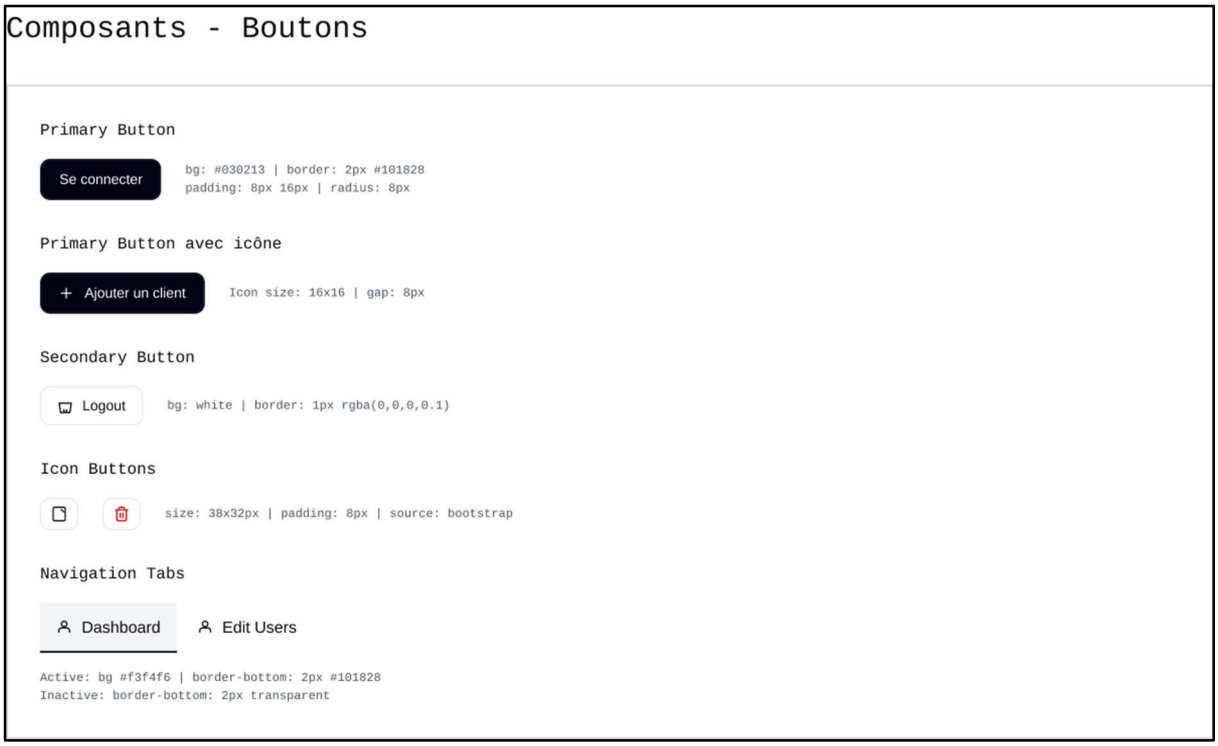
Input Placeholder - 14px
font-family: Arimo | font-size: 14px | color: #717182

Design 2 : Design System & Style Guide, Typographie

Composants — Boutons :

Cinq types de boutons couvrent l'ensemble des interactions. Le bouton primaire (fond sombre, fort contraste) représente l'action principale de la page. Sa variante avec icône est utilisée pour les actions de création (« + Ajouter un client »). Le bouton secondaire (fond blanc, bordure légère) est réservé aux actions moins prioritaires comme la déconnexion.

Les boutons icônes (crayon, corbeille) permettent les actions contextuelles dans les tableaux, la corbeille utilisant une bordure rouge pour signaler le caractère destructif. Les onglets de navigation distinguent l'onglet actif par une bordure inférieure sombre, permettant à l'utilisateur de toujours se situer dans l'application.



Design 3 : Design System & Style Guide, Composants – Boutons

Composants — Badges :

Les badges sont des indicateurs visuels colorés qui catégorisent les informations dans les tableaux. Trois variantes ont été définies : vert pour les états positifs (online, allow), rouge pour les états restrictifs (deny), et gris pour les informations descriptives neutres (type de règle, statut offline).

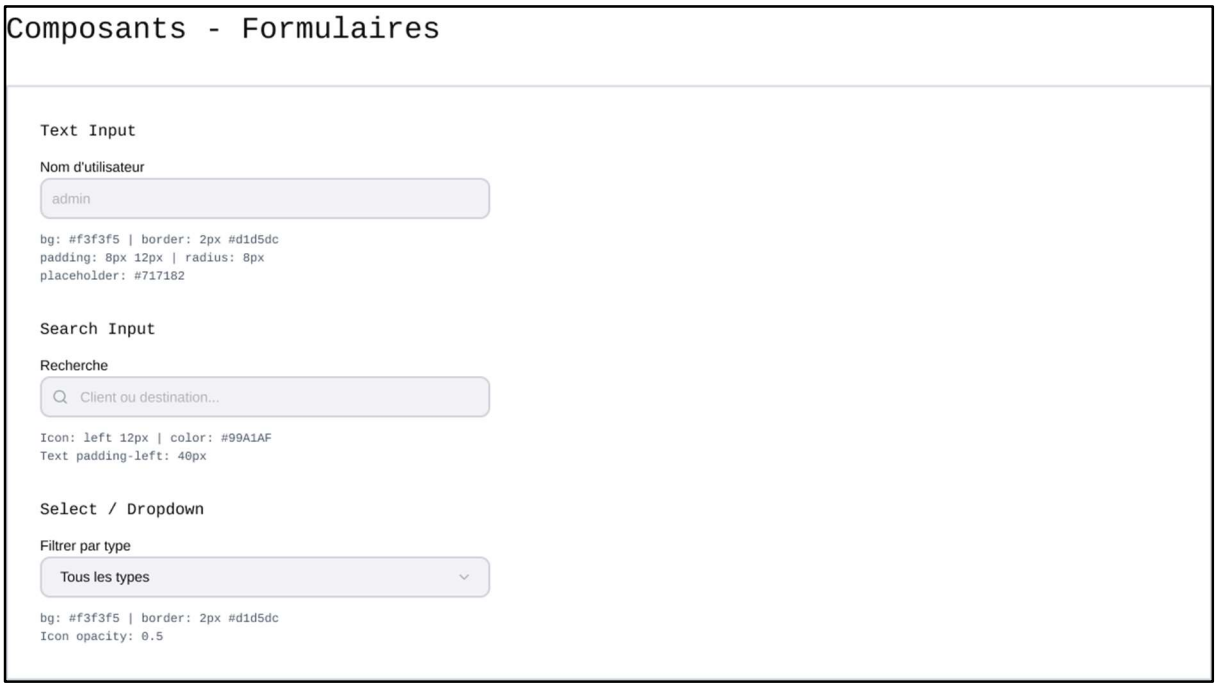
Ce système permet à l'administrateur de balayer visuellement un tableau et de comprendre la situation sans lire chaque cellule en détail.

Design 4 : Design System & Style Guide, Composants – Badges

Composants — Formulaires :

Trois types de champs ont été définis : le champ texte standard (login, formulaires), le champ de recherche enrichi d'une icône loupe, et le menu déroulant pour les

filtres. Les trois partagent le même style visuel (fond, bordure, rayon), ce qui renforce la cohérence et réduit la charge cognitive de l'utilisateur.



Design 5 : Design System & Style Guide, Composants – Formulaires

Composants — Cartes & Tableaux :

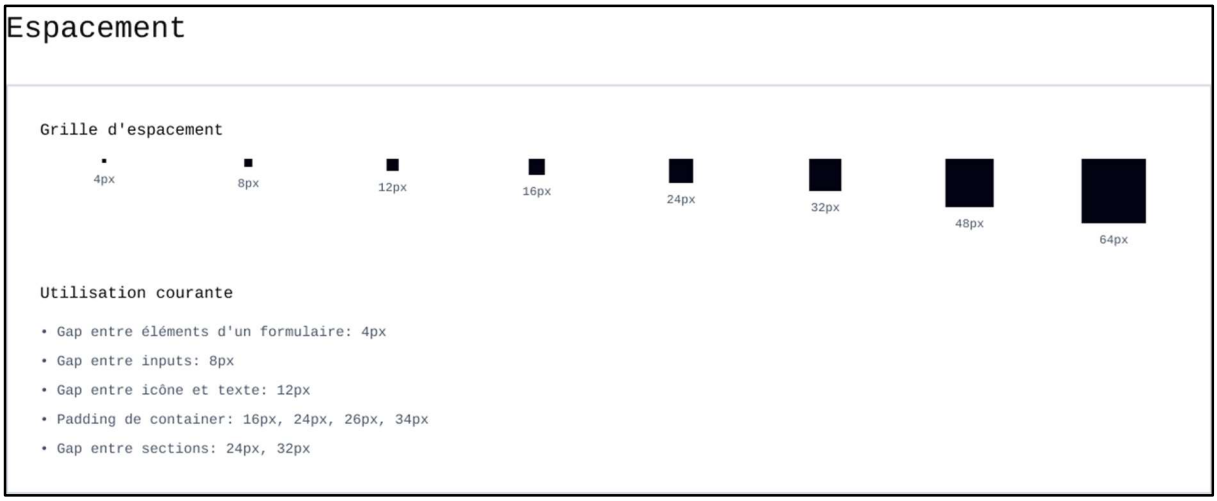
La carte statistique affiche les indicateurs clés du Dashboard avec une structure simple : icône, label descriptif et valeur numérique en grand. Elle permet une lecture immédiate des chiffres importants.

Le tableau est le composant central de l'application, présent sur les trois pages. L'en-tête se distingue par un fond légèrement grisé, et un padding de 16px afin d'avoir un espacement confortable pour la lecture des adresses IP et des noms de domaine.

Design 6 : Design System & Style Guide, Composants – Cartes & Tableaux

Composants — Espacement :

La grille d'espacement repose sur des multiples de 4px (4, 8, 12, 16, 24, 32, 48, 64px), une convention répandue dans le design d'interfaces. Chaque valeur a un usage précis : 4px entre éléments de formulaire, 8px entre inputs, 12px entre icône et texte, 16-34px pour le padding des containers, 24-32px entre sections. Cette grille garantit un rythme visuel régulier et évite les espacements arbitraires.



Design 7 : Design System & Style Guide, Espacement

Composants — Bordures & Coins arrondis :

Deux épaisseurs de bordure sont utilisées : 1px pour les éléments légers (tableaux, badges) et 2px pour les éléments structurants (cartes, inputs, containers). Deux rayons de coins arrondis sont appliqués : 8px pour les éléments interactifs (boutons, inputs, cartes) et 0px pour les éléments structurels (tableaux, containers). Ce contraste aide l'utilisateur à distinguer intuitivement les zones cliquables des zones de consultation.

Design 8 : Design System & Style Guide, Bordures & Coins arrondis

Composants — Icônes :

Les icônes proviennent de Bootstrap Icons, une bibliothèque open source offrant un large catalogue d'icônes adaptées aux interfaces d'administration. Quatre tailles sont définies selon le contexte : 16px (boutons), 20px (cartes), 32px (header) et 40px (logo). Le style retenu est le « line icon » (trait sans remplissage), plus léger visuellement et cohérent avec l'esthétique épurée de l'application.

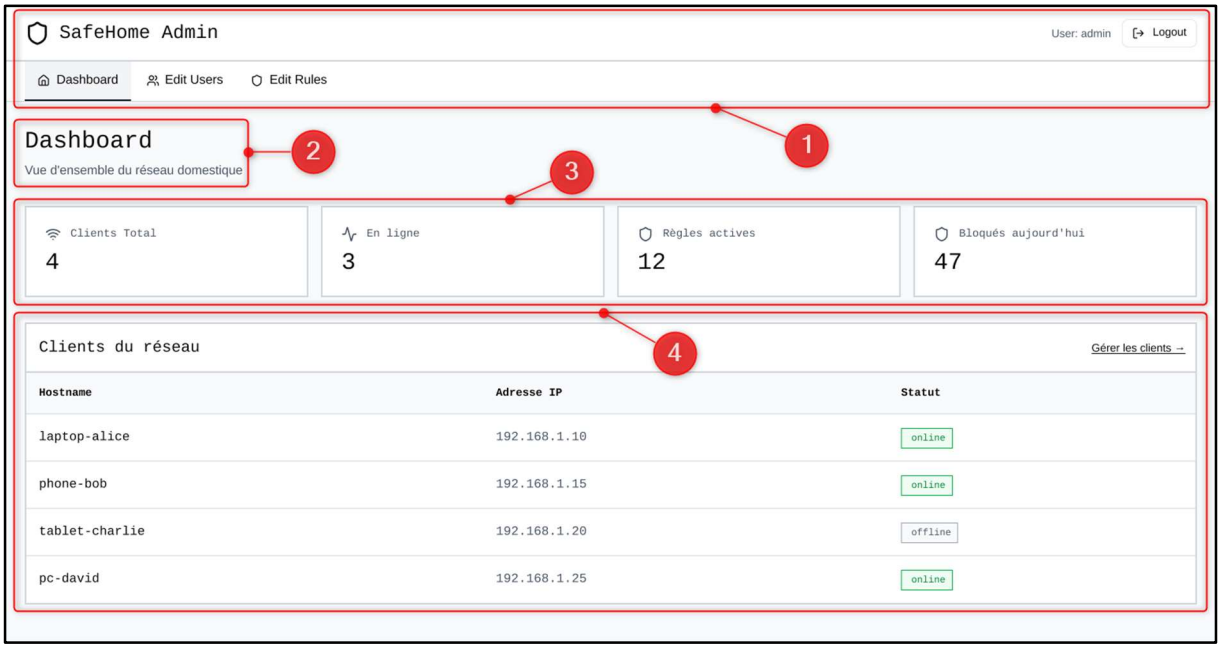
icons.getbootstrap.com TODO, mettre un lien dans la webographie

Design 9 : Design System & Style Guide, Icônes

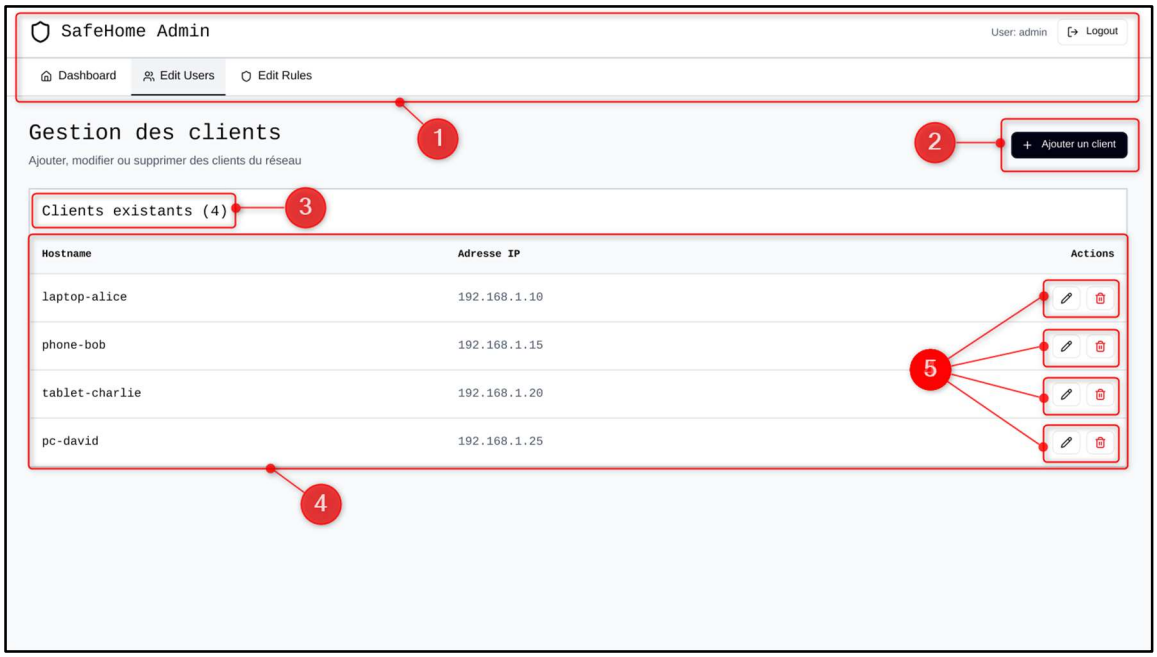
Maquette :

En prenant en compte tous les éléments définis dans les sections précédentes (de « Approche de conception » jusqu'à « Design System et Style Guide »), les maquettes basse fidélité ont été réalisées dans Figma. Elles représentent la structure et la disposition de chaque page de l'application, en intégrant directement les composants, les couleurs et la typographie du Design System. Ces maquettes ont servi de référence constante durant le développement pour garantir une implémentation fidèle à la conception initiale. L'application comporte quatre écrans : une page de connexion et trois pages principales accessibles via la navigation par onglets.

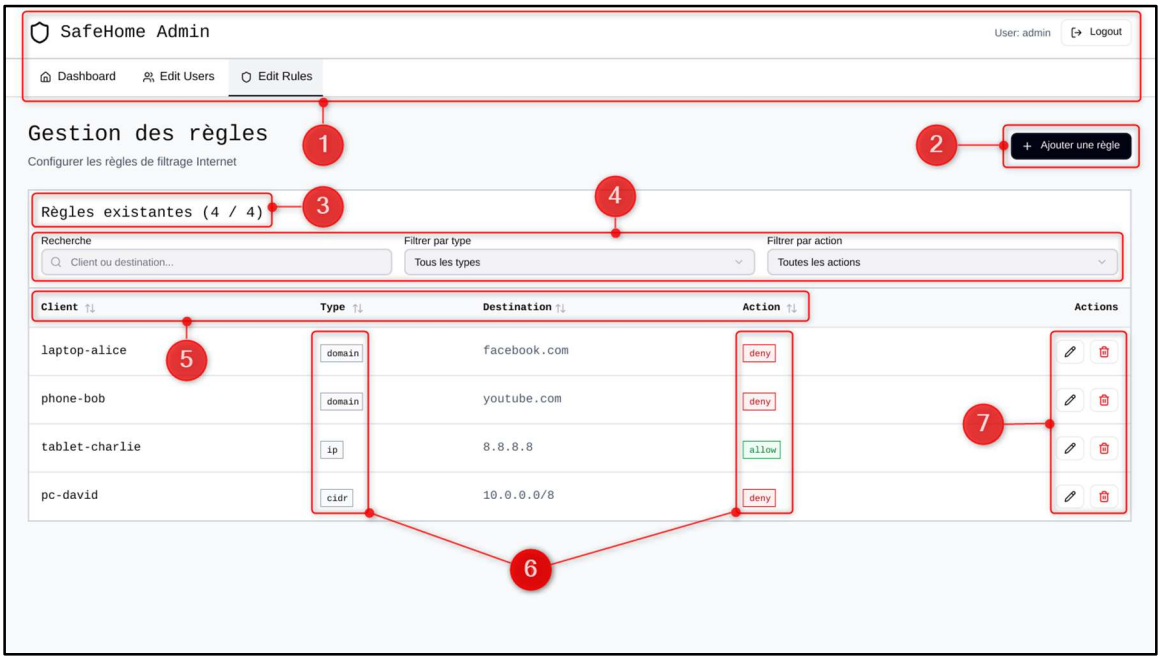
Design 10 : Maquette, Login



Design 11 : Maquette, Dashboard



Design 12 : Maquette, Edit Users



Design 13 : Maquette, Edit Rules

Page Login

N°	Élément	Description

1	**Logo et identité**	Logo SafeHome avec le nom de l'application et le sous-titre « Administration du routeur familial », identifie immédiatement le contexte de l'application.
2	**Carte de connexion**	Conteneur centré regroupant le formulaire, séparé visuellement par un espacement clair.
3	**Champs de saisie**	Deux champs (nom d'utilisateur et mot de passe) utilisant le composant Text Input du Design System, avec placeholders pour guider l'utilisateur.
4	**Bouton « Se connecter »**	Bouton primaire pleine largeur, seule action possible sur cette page, ce qui élimine toute ambiguïté.

Page Dashboard

N°	Élément	Description
1	**En-tête commun**	Logo SafeHome, onglets de navigation (Dashboard actif), nom d'utilisateur et bouton Logout. Présent sur toutes les pages authentifiées.
2	**Titre et sous-titre**	« Dashboard » avec « Vue d'ensemble du réseau domestique », contextualise immédiatement le contenu.
3	**Cartes statistiques**	Quatre indicateurs clés côte à côte : Clients Total, En ligne, Règles actives, Bloqués aujourd'hui. Permettent un état des lieux instantané du réseau.
4	**Tableau des clients**	Liste des appareils avec hostname, adresse IP et statut online/offline via badges colorés. Le lien « Gérer les clients → » redirige vers Edit Users.

Page Edit Users

N°	Élément	Description
1	**En-tête commun**	Onglet « Edit Users » actif, identifiable par la bordure inférieure sombre.
2	**Bouton « + Ajouter »**	Bouton primaire avec icône, positionné en haut à droite pour un accès rapide à l'action principale.
3	**Compteur**	« Clients existants (4) » indique le nombre total de clients enregistrés.
4	**Tableau des clients**	Colonnes Hostname et Adresse IP en police monospace pour l'alignement des données réseau.
5	**Boutons d'action**	Icône crayon (édition) et icône corbeille rouge (suppression) sur chaque ligne.

Page Edit Rules

N°	Élément	Description
1	**En-tête commun**	Onglet « Edit Rules » actif.
2	**Bouton « + Ajouter »**	Accès rapide à la création d'une nouvelle règle de filtrage.
3	**Compteur dynamique**	« Règles existantes (4 / 4) » s'adapte en fonction des filtres appliqués.
4	**Recherche et filtres**	Trois éléments côte à côte : recherche textuelle, filtre par type (domain/ip/cidr) et filtre par action (deny/allow).
5	**En-têtes triables**	Les colonnes Client, Type, Destination et Action possèdent des icônes de tri (↑↓).
6	**Badges type et action**	Badges gris pour le type (domain, ip, cidr), rouge pour deny et vert pour allow.
7	**Boutons d'action**	Même pattern que Edit Users : crayon pour l'édition, corbeille rouge pour la suppression.

2.5 Stratégie de test

Approche générale :

La stratégie de test du projet SafeHome suit une approche ascendante (bottom-up) : les composants les plus bas de l'infrastructure sont testés en premier, car chaque couche supérieure dépend du bon fonctionnement de la précédente. Il serait inutile de tester le filtrage iptables si le routage NAT ne fonctionne pas, ou de tester l'application web si la base de données n'est pas accessible. Les tests sont donc organisés en cinq phases successives, chacune validant un niveau de l'architecture avant de passer au suivant.

Types de tests et ordre d'exécution :

Phase 1 - Infrastructure réseau et connectivité (tests système).

Cette première phase valide que les machines virtuelles sont correctement configurées dans VirtualBox, que les interfaces réseau sont actives et que les VM peuvent communiquer entre elles via le réseau interne. Les tests consistent en des commandes réseau de base : vérification des interfaces (ip addr), ping entre les VM, et vérification de l'accès Internet depuis VM2 via le bridge. C'est le socle sur lequel repose tout le reste.

Phase 2 - Services réseau (tests fonctionnels unitaires).

Une fois la connectivité établie, chaque service est testé individuellement. Le serveur DHCP est validé en vérifiant qu'un client obtient une adresse IP dans la plage attendue avec les bons paramètres (gateway, DNS, domaine, durée du bail). Le serveur DNS est testé par des requêtes nslookup pour la résolution directe (safehome.local) et les forwarders (résolution de domaines externes). Le

NAT/MASQUERADE est vérifié en testant l'accès Internet depuis un client du réseau interne. MySQL est testé par une connexion et une vérification des tables.

Phase 3 - Scripts Python (tests d'intégration).

Les scripts sont testés après les services car ils dépendent à la fois de MySQL et d'iptables. sync_firewall.py est validé en créant une règle en base, en exécutant le script manuellement, puis en vérifiant que la règle apparaît dans iptables. Le test inverse (suppression) est également effectué. monitor.py est testé en vérifiant qu'après exécution, la table monitoring contient les statuts des clients et que la table blocked_traffic enregistre les blocages détectés dans syslog.

Phase 4 - Application web (tests fonctionnels et interface).

L'application est testée couche par couche : d'abord la connexion à MySQL depuis ASP.NET (Entity Framework), puis chaque fonctionnalité CRUD (clients et règles), l'authentification (login avec BCrypt, protection des routes, logout), et enfin l'affichage du dashboard avec les données de monitoring. Les tests couvrent à la fois le backend (les contrôleurs retournent les bonnes données) et le frontend (les pages s'affichent correctement, les formulaires fonctionnent, les filtres et la recherche répondent).

Phase 5 - Tests de bout en bout (end-to-end).

La dernière phase teste le scénario complet : un administrateur se connecte à l'interface depuis le PC hôte, crée un client et une règle de blocage (par exemple facebook.com), puis vérifie depuis la VM Client que le site est effectivement inaccessible. Le dashboard doit refléter le blocage après l'exécution du cycle monitor.py. Ce test valide l'intégration de toute la chaîne : interface web → base de données → sync_firewall.py → iptables → blocage réel → monitor.py → dashboard.

<u>Moyens à mettre en oeuvre :</u>

L'environnement de test est l'infrastructure elle-même : les trois machines virtuelles VirtualBox (VM1, VM2, VM Client) exécutées sur le PC hôte Windows 10. Les tests réseau utilisent les outils Linux standards disponibles sur les VM : ping, ip, nslookup, curl, iptables -L, mysql. Les tests de l'application web sont effectués via un navigateur depuis le PC hôte (accès via l'IP bridge de VM2). Les scripts Python sont exécutés manuellement en ligne de commande pour observer leur sortie avant d'être confiés au cron. Aucun framework de test automatisé n'est utilisé ; tous les tests sont manuels et documentés par des captures d'écran du terminal ou du navigateur.

<u>Couverture des tests :</u>

Les tests ne sont pas exhaustifs. Dans le cadre du TPI, la priorité est donnée aux scénarios nominaux (le cas normal qui doit fonctionner) et aux cas d'erreur les plus critiques (mot de passe incorrect au login, règle sur un domaine inexistant, client hors ligne). Les cas limites moins probables (saturation de la plage DHCP, fichier

syslog corrompu, perte de connexion MySQL en cours de synchronisation) ne sont pas systématiquement testés par manque de temps.

Cette approche est justifiée par le contexte du projet : il s'agit d'un prototype fonctionnel destiné à démontrer la faisabilité du concept, pas d'un produit en production. La couverture se concentre donc sur les fonctionnalités exigées par le cahier des charges et sur les scénarios que les experts évalueront lors de la présentation.

Données de test :

Les données de test sont des données réalistes mais fictives, créées spécifiquement pour le projet. Elles comprennent un compte administrateur (admin / mot de passe hashé en BCrypt), un client simulant des appareils du réseau domestique (la VM Client avec le hostname « Client-Test » et une adresse obtenue par DHCP), et des règles de filtrage couvrant les trois types supportés : domaine (par exemple facebook.com, youtube.com), adresse IP unique, et plage CIDR. Les domaines choisis pour les tests de blocage sont des sites réels facilement vérifiables (on peut constater immédiatement si l'accès est bloqué ou non depuis le navigateur de la VM Client).

Testeurs extérieurs :

Le projet ne prévoit pas de testeurs extérieurs formels durant la phase de développement. Cependant, lors de la présentation du TPI, les experts joueront le rôle de testeurs en vérifiant le bon fonctionnement de l'infrastructure en conditions réelles. Pour anticiper cette évaluation, les tests de bout en bout (phase 5) sont conçus pour reproduire les scénarios que les experts sont susceptibles de demander : ajouter un client, créer une règle, vérifier le blocage effectif, consulter le dashboard.

2.6 Risques techniques

monitor.py - Monitoring et détection des blocages :

Objectif et logique :

- *risques techniques (complexité, manque de compétences, …).*

Décrire aussi quelles solutions ont été appliquées pour réduire les risques (priorités, formation, actions, …).

2.7 Dossier de conception

Fournir tous les document de conception:

- *le choix du matériel HW*
- *le choix des systèmes d'exploitation pour la réalisation et l'utilisation*
- *le choix des outils logiciels pour la réalisation et l'utilisation*
- *site web: réaliser les maquettes avec un logiciel, décrire toutes les animations sur papier, définir les mots-clés, choisir une formule d'hébergement, définir la méthode de mise à jour, …*
- *bases de données: décrire le modèle relationnel, le contenu détaillé des tables (caractéristiques de chaque champs) et les requêtes.*
- *programmation et scripts: organigramme, architecture du programme, découpage modulaire, entrées-sorties des modules, pseudo-code / structogramme…*

Le dossier de conception devrait permettre de sous-traiter la réalisation du projet !

3 Réalisation

3.1 Dossier de réalisation

Décrire la réalisation "physique" de votre projet

- *les répertoires où le logiciel est installé*
- *la liste de tous les fichiers et une rapide description de leur contenu (des noms qui parlent !)*
- *les versions des systèmes d'exploitation et des outils logiciels*
- *la description exacte du matériel*
- *le numéro de version de votre produit !*
- *programmation et scripts: librairies externes, dictionnaire des données, reconstruction du logiciel - cible à partir des sources.*

NOTE : Evitez d'inclure les listings des sources, à moins que vous ne désiriez en expliquer une partie vous paraissant importante. Dans ce cas n'incluez que cette partie…

3.2 Description des tests effectués

N°	Phase	Composant testé	Description du test	Pré-conditions	Résultat attendu	Résultat obtenu	Réussite	Commentaire / Preuve
1	1	VirtualBox - VM1	Vérifier que l'interface enp0s3 est présente et en état UP	VM1 démarrée, Adapter 1 = Internal Network (intnet-lan)	ip link show affiche enp0s3 state UP	Voir Capture X	Non testé	Capture terminal
2	1	VirtualBox - VM2	Vérifier que les interfaces enp0s3 et enp0s8 sont présentes	VM2 démarrée, Adapter 1 = Internal Network, Adapter 2 = Bridged	ip link show affiche enp0s3 et enp0s8 en state UP		Non testé	Capture terminal
3	1	VirtualBox - VM Client	Vérifier que l'interface enp0s3 est présente	VM Client démarrée, Adapter 1 = Internal Network (intnet-lan)	ip link show affiche enp0s3 state UP		Non testé	Capture terminal
4	1	VM1 - IP statique	Vérifier l'adresse IP statique de VM1	Netplan configuré et appliqué	ip addr show enp0s3 affiche 192.168.15.254/24		Non testé	Capture terminal
5	1	VM2 - IP statique (enp0s3)	Vérifier l'adresse IP statique de VM2 côté LAN	Netplan configuré et appliqué	ip addr show enp0s3 affiche 192.168.15.1/24		Non testé	Capture terminal
6	1	VM2 - IP dynamique (enp0s8)	Vérifier que VM2 obtient une IP du FAI via bridge	Câble réseau/Wi-Fi connecté sur PC hôte	ip addr show enp0s8 affiche une IP du réseau FAI (ex: 192.168.1.x)		Non testé	Capture terminal
7	1	Connectivité VM1 avec VM2	Ping entre VM1 et VM2 via le réseau interne	IP statiques configurées sur les deux VM	ping 192.168.15.1 depuis VM1 → réponse OK ping 192.168.15.254 depuis VM2 → réponse OK		Non testé	Capture terminal
8	1	VM2 - Accès Internet	Vérifier que VM2 accède à Internet via le bridge	enp0s8 configurée en Bridged Adapter	ping 8.8.8.8 depuis VM2 → réponse OK		Non testé	Capture terminal

9	2	VM1 - DHCP	Vérifier que la VM Client obtient une IP via DHCP	isc-dhcp-server actif sur VM1, VM Client en Internal Network	VM Client reçoit IP entre 192.168.15.100-200, gateway=192.168.15.1, DNS=192.168.15.254		Non testé	ip addr + ip route + cat /etc/resolv.conf
10	2	VM1 - DHCP (bail)	Vérifier la durée du bail DHCP	VM Client connectée au réseau	Bail de 6 heures (21600 secondes)		Non testé	dhclient -v ou fichier lease
11	2	VM1 - DHCP (domaine)	Vérifier le domaine attribué par DHCP	VM Client connectée au réseau	Domaine = safehome.local dans resolv.conf		Non testé	cat /etc/resolv.conf
12	2	VM1 - DNS (résolution locale)	Résoudre un nom interne via BIND9	BIND9 actif sur VM1, zone safehome.local configurée	nslookup gateway.safehome.local 192.168.15.254 → 192.168.15.1		Non testé	Capture terminal
13	2	VM1 - DNS (forwarder)	Résoudre un nom externe via les forwarders	BIND9 avec forwarders 8.8.8.8 configurés	nslookup google.com 192.168.15.254 → résolution OK		Non testé	Capture terminal
14	2	VM1 - DDNS	Vérifier l'enregistrement DNS dynamique du client	DHCP + BIND9 configurés pour DDNS, VM Client connectée	nslookup client-test.safehome.local → IP du client		Non testé	Capture terminal
15	2	VM2 - NAT/MASQUERADE	Vérifier que le client accède à Internet via VM2	IP forwarding activé, règle MASQUERADE en place	ping 8.8.8.8 depuis VM Client → réponse OK		Non testé	Capture terminal
16	2	VM2 - NAT (DNS externe)	Vérifier la résolution DNS + accès web depuis le client	NAT + DNS fonctionnels	ping google.com depuis VM Client → réponse OK curl http://example.com → contenu HTML reçu		Non testé	Capture terminal

17	2	VM2 - MySQL	Vérifier la connexion MySQL avec l'utilisateur applicatif	MySQL installé, utilisateur firewall_admin créé	mysql -u firewall_admin -p → connexion OK SHOW TABLES FROM db_firewall → tables listées		Non testé	Capture terminal
18	2	VM2 - MySQL (tables)	Vérifier la structure des tables principales	Base db_firewall créée	Tables clients, firewall_rules, admin_users existent avec les colonnes attendues		Non testé	DESCRIBE <table>
19	3	sync_firewall.py - Ajout	Ajouter une règle en base puis exécuter le script	MySQL accessible, chaîne FIREWALL_RULES créée	iptables -L FIREWALL_RULES affiche la règle DROP correspondante		Non testé	iptables -L -n avant/après
20	3	sync_firewall.py - Suppression	Supprimer une règle en base puis exécuter le script	Règle existante en base et dans iptables	La règle disparaît d'iptables après exécution		Non testé	iptables -L -n avant/après
21	3	sync_firewall.py - Domaine	Tester la résolution DNS d'un domaine dans une règle	Règle de type "domain" (ex: facebook.com) en base	Le script résout le domaine en IPs et crée les règles DROP pour chaque IP		Non testé	Sortie console + iptables -L
22	3	sync_firewall.py - LOG	Vérifier l'insertion des règles LOG [FW_BLOCKED]	Règle DROP en base, script exécuté	Chaque règle DROP est précédée d'une règle LOG avec préfixe [FW_BLOCKED]		Non testé	iptables -L FIREWALL_RULES -n
23	3	sync_firewall.py - Cron	Vérifier l'exécution automatique par cron	Entrée crontab configurée (*/5 * * * *)	Le script s'exécute toutes les 5 minutes (vérifiable dans les logs)		Non testé	crontab -l + journalctl ou syslog
24	3	monitor.py - Ping clients	Vérifier la détection online/offline des clients	VM Client démarrée et connectée	Table monitoring contient une entrée avec is_online=1 pour le client		Non testé	SELECT * FROM monitoring

25	3	monitor.py - Client offline	Vérifier la détection d'un client hors ligne	VM Client éteinte	Table monitoring contient une entrée avec is_online=0		Non testé	SELECT * FROM monitoring
26	3	monitor.py - Trafic bloqué	Vérifier l'enregistrement du trafic bloqué	Règle de blocage active, tentative d'accès depuis VM Client	Table blocked_traffic contient des entrées avec src_ip, dst_ip, domain, hit_count		Non testé	SELECT * FROM blocked_traffic
27	3	monitor.py - Reverse DNS	Vérifier la résolution du domaine pour les IPs bloquées	Blocage d'un domaine connu (ex: facebook.com)	Colonne domain renseignée dans blocked_traffic (ex: "facebook.com")		Non testé	SELECT dst_ip, domain FROM blocked_traffic
28	3	monitor.py - Pas de doublons	Vérifier que les anciennes lignes syslog ne sont pas réinsérées	Script déjà exécuté une fois, aucun nouveau blocage	Aucune nouvelle entrée dans blocked_traffic (fichier de position fonctionnel)		Non testé	Compter les lignes avant/après
29	3	monitor.py - Cron	Vérifier l'exécution automatique par cron	Entrée crontab configurée (*/2 * * * *)	Le script s'exécute toutes les 2 minutes		Non testé	crontab -l + journalctl ou syslog
30	4	Kestrel - Démarrage	Vérifier que l'application démarre et écoute sur le port 5000	Application déployée, service systemd actif	curl http://192.168.15.1:5000 → redirection vers /Auth/Login (code 302)		Non testé	curl -v ou navigateur
31	4	Accès depuis PC hôte	Accéder à l'application depuis le navigateur du PC hôte	VM2 bridge actif, application en écoute	Page de login affichée dans le navigateur à http://<IP-bridge>:5000		Non testé	Capture navigateur
32	4	Auth - Login valide	Se connecter avec des identifiants corrects	Utilisateur admin créé en base avec hash BCrypt	Redirection vers le Dashboard, cookie d'authentification créé		Non testé	Capture navigateur
33	4	Auth - Login invalide	Se connecter avec un mot de passe incorrect	Page de login affichée	Message d'erreur "Nom d'utilisateur ou mot de passe incorrect." affiché		Non testé	Capture navigateur

34	4	Auth - Protection des routes	Accéder au dashboard sans être connecté	Session non authentifiée (ou cookie expiré)	Redirection automatique vers /Auth/Login		Non testé	Capture navigateur
35	4	Auth - Logout	Se déconnecter depuis l'interface	Session authentifiée active	Redirection vers la page de login, cookie supprimé		Non testé	Capture navigateur
36	4	Dashboard - Affichage	Vérifier l'affichage des 4 cartes de statistiques	Données de monitoring en base, utilisateur connecté	Cartes affichent : Clients total, En ligne, Règles actives, Bloqués aujourd'hui		Non testé	Capture navigateur
37	4	Dashboard - Statut clients	Vérifier l'affichage du statut online/offline	monitor.py exécuté au moins une fois	Tableau des clients avec indicateur vert (online) ou rouge (offline)		Non testé	Capture navigateur
38	4	Clients - Liste	Afficher la liste des clients enregistrés	Au moins un client en base	Page Edit Users affiche les clients avec hostname et IP		Non testé	Capture navigateur
39	4	Clients - Ajout	Ajouter un nouveau client via le formulaire modal	Page Edit Users ouverte	Client ajouté en base, apparaît dans la liste, message de succès affiché		Non testé	Capture navigateur + vérif DB
40	4	Clients - Modification	Modifier le hostname d'un client existant	Client existant en base	Hostname mis à jour en base et dans la liste		Non testé	Capture navigateur
41	4	Clients - Suppression	Supprimer un client et ses règles associées	Client avec règles existantes	Client supprimé, ses règles supprimées en cascade		Non testé	Capture navigateur + vérif DB
42	4	Règles - Liste	Afficher la liste des règles de filtrage	Au moins une règle en base	Page Edit Rules affiche les règles avec type, destination, action		Non testé	Capture navigateur
43	4	Règles - Ajout (domaine)	Ajouter une règle de type domaine	Client existant en base	Règle créée avec type=domain, destination=facebook.com, action=deny		Non testé	Capture navigateur + vérif DB

44	4	Règles - Ajout (IP)	Ajouter une règle de type IP	Client existant en base	Règle créée avec type=ip, destination=1.2.3.4		Non testé	Capture navigateur + vérif DB
45	4	Règles - Ajout (CIDR)	Ajouter une règle de type CIDR	Client existant en base	Règle créée avec type=cidr, destination=10.0.0.0/8		Non testé	Capture navigateur + vérif DB
46	4	Règles - Modification	Modifier la destination d'une règle existante	Règle existante en base	Destination mise à jour en base et dans la liste		Non testé	Capture navigateur
47	4	Règles - Suppression	Supprimer une règle de filtrage	Règle existante en base	Règle supprimée de la base, disparaît de la liste		Non testé	Capture navigateur + vérif DB
48	4	Règles - Filtres	Filtrer les règles par type (domaine/IP/CIDR) et par action (deny/allow)	Plusieurs règles de types différents en base	La liste se filtre dynamiquement, compteur de résultats mis à jour		Non testé	Capture navigateur
49	4	Règles - Recherche	Rechercher une règle par destination avec l'autocomplétion	Plusieurs règles en base	L'autocomplétion propose des suggestions, les résultats correspondent		Non testé	Capture navigateur
50	5	Scénario complet - Blocage	Créer une règle via l'interface → vérifier le blocage effectif	1. Login admin 2. Client enregistré 3. sync_firewall.py actif (cron)	Après création de la règle et exécution de sync : curl facebook.com depuis VM Client → timeout/bloqué		Non testé	Captures navigateur + terminal
51	5	Scénario complet - Déblocage	Supprimer la règle via l'interface → vérifier l'accès rétabli	Règle de blocage active dans iptables	Après suppression et exécution de sync : curl facebook.com depuis VM Client → réponse OK		Non testé	Captures navigateur + terminal
52	5	Scénario complet - Dashboard	Vérifier que le dashboard reflète les blocages	Blocage effectué, monitor.py exécuté	Carte "Bloqués aujourd'hui" incrémentée, domaine affiché dans les détails		Non testé	Capture navigateur

53	5	Accès externe - Téléphone	Accéder à l'application depuis un téléphone sur le même réseau Wi-Fi	VM2 bridge actif, téléphone sur le même réseau que le PC hôte	Page de login affichée sur le téléphone à http://<IP-bridge>:5000		Non testé	Capture téléphone
54	5	Persistance - Redémarrage VM2	Redémarrer VM2 et vérifier que les services reprennent	Tous les services configurés en démarrage auto	Après reboot : MySQL actif, Kestrel actif, iptables restaurées, cron actif		Non testé	systemctl status + iptables -L
55	5	Persistance - Redémarrage VM1	Redémarrer VM1 et vérifier DHCP + DNS	Services en démarrage auto	Après reboot : isc-dhcp-server actif, BIND9 actif, client peut renouveler bail		Non testé	systemctl status + dhclient -v

3.3 Erreurs restantes

S'il reste encore des erreurs:

- *Description détaillée*
- *Conséquences sur l'utilisation du produit*
- *Actions envisagées ou possibles*

3.4 Liste des documents fournis

Lister les documents fournis au client avec votre produit, en indiquant les numéros de versions

- *le rapport de projet*
- *le manuel d'Installation (en annexe)*
- *le manuel d'Utilisation avec des exemples graphiques (en annexe)*
- *autres…*

4 Conclusions

Développez en tous cas les points suivants:

- *Objectifs atteints / non-atteints*
- *Points positifs / négatifs*
- *Difficultés particulières*
- *Suites possibles pour le projet (évolutions & améliorations)*

5 Annexes

5.1 Résumé du rapport du TPI version succincte de la documentation

5.2 Glossaire

M

Méthodologie · 6

T

TPI · **3**

5.3 Webographie

Consulté le 06.02.2026 :
- Frost, Brad Atomic Web Design, https://bradfrost.com/blog/post/atomic-web-design/
- [YTB] SJ Abgrall, Glossaire word, https://www.youtube.com/watch?v=srlNOQmQ4Ig
- CIFC-VD, Méthode des 6 étapes, https://www.cifc-vd.ch/docs/2_1/taches/la_methode_des_6_etapes.pdf
- AFCI-JU, Méthode des 6 étapes, https://www.afci-ju.ch/fichiers/Planification-par-la-mthode-des-6-tapes-15.pdf
- [YTB] Minh Nguyen, Page orienté de manière horizosntale, https://www.youtube.com/watch?v=JvHbK06IP90

Consulté le 09.02.2026 :
- Oracle, réseau VirtualBox, https://docs.oracle.com/en/virtualization/virtualbox/6.0/user/network_nat.html
- Oracle, réseau VirtualBox, https://docs.oracle.com/en/virtualization/virtualbox/6.0/user/network_internal.html
- Nakivo, réseau VirtualBox, https://www.nakivo.com/blog/virtualbox-network-setting-guide/
- [YTB] IT-Connect – Florian, réseau VirtualBox, https://www.youtube.com/watch?v=iPPKNc3appk
- Ubuntu, IP statiques Netplan, https://documentation.ubuntu.com/server/explanation/networking/configuring-networks/
- netplan, IP statiques Netplan, https://netplan.readthedocs.io/en/stable/using-static-ip-addresses/
- linuxconfig, IP statiques Netplan, https://linuxconfig.org/setting-a-static-ip-address-in-ubuntu-24-04-via-the-command-line
- [YTB] ThatLinuxDude, IP statiques Netplan, https://www.youtube.com/watch?v=QLgEh8FHjwc
- [YTB] SavvyNik, IP statiques Netplan, https://www.youtube.com/watch?v=YZicKxPmCos
- revsys, Routage NAT iptables, https://www.revsys.com/writings/quicktips/nat.html
- malekal, Routage NAT iptables, https://www.malekal.com/utiliser-masquerade-iptables-nat/
- netfilter, Routage NAT iptables, https://www.netfilter.org/documentation/HOWTO/fr/NAT-HOWTO-6.html
- [YTB] Andrei Dumitrescu, Routage NAT iptables, https://www.youtube.com/watch?v=dUsGv3BZ_Uc
- it-connect, Routage NAT iptables, https://www.it-connect.fr/chapitres/netfilter-nftables-ca-fonctionnement-comment/
- [YTB] LJP-Infos, Routage NAT iptables, https://www.youtube.com/watch?v=fLHFPgFR_kk
- Ubuntu, Serveur DHCP (isc-dhcp-server), https://documentation.ubuntu.com/server/how-to/networking/install-isc-dhcp-server/
- Ubuntu, Serveur DHCP (isc-dhcp-server), https://help.ubuntu.com/community/isc-dhcp-server
- linuxfordevices, Serveur DHCP (isc-dhcp-server), https://www.linuxfordevices.com/tutorials/ubuntu/dhcp-server-on-ubuntu
- [YTB] Linux Tech School, Serveur DHCP (isc-dhcp-server), https://www.youtube.com/watch?v=Hj5d1eu5fpQ

Consulté le 11.02.2026 :
- digitalocean, DNS Bind9, https://www.digitalocean.com/community/tutorials/how-to-configure-bind-as-a-private-network-dns-server-on-ubuntu-20-04
- linuxtechi, DNS Bind9, https://www.linuxtechi.com/install-configure-bind-9-dns-server-ubuntu-debian/
- linuxfordevices, DNS Bind9, https://www.linuxbabe.com/ubuntu/set-up-local-dns-resolver-ubuntu-20-04-bind9
- [YTB] tubetek tutorials, DNS Bind9, https://www.youtube.com/watch?v=ZKnclk4atZ0
- Ubuntu, MySQL, https://documentation.ubuntu.com/server/how-to/databases/install-mysql/
- [YTB] ProgrammingKnowledge, MySQL, https://www.youtube.com/watch?v=455KKhZyvow

Consulté le 12.02.2026 :
- [YTB] Susan B., VsCode Remote SSH, https://www.youtube.com/watch?v=cOopQQIL8JU
- visualstudio, VsCode Remote SSH, https://code.visualstudio.com/docs/remote/ssh
- askubuntu, VsCode Remote SSH, https://askubuntu.com/questions/51925/how-do-i-configure-a-new-ubuntu-installation-to-accept-ssh-connections
- dedicatedcore, VsCode Remote SSH, https://www.dedicatedcore.com/blog/enable-ssh-ubuntu/
- [YTB] raksrahul., Iptables et python, https://www.youtube.com/watch?v=n5wTgdxkuok
- digitalocean., Iptables et python, https://www.digitalocean.com/community/tutorials/how-to-implement-a-basic-firewall-template-with-iptables-on-ubuntu-20-04

Consulté le 13.02.2026 :
- ubuntu, Cron, https://doc.ubuntu-fr.org/cron
- serverspace, Cron, https://serverspace.io/fr/articles/setup-cron-automate-tasks-ubuntu/

Consulté le 14.02.2026 :

5.4 Journal de travail

Date	Durée	Activité	Remarques

5.5 Manuel d'Installation

5.6 Manuel d'Utilisation

5.7 Archives du projet

Media, … dans une fourre en plastique